UNIVERSIDAD POLITÉCNICA DE MADRID

E.T.S. DE INGENIERÍA DE SISTEMAS INFORMÁTICOS

PROYECTO FIN DE GRADO

GRADO EN INGENIERÍA DEL SOFTWARE

Sistema de préstamos descentralizado basado en la red blockchain de Ethereum

Desarrollado por: Aquilino Pascual Sanz

Dirigido por: Alberto Díaz Álvarez y Guillermo Sánchez Gutiérrez-Cabello

Madrid, 28 de octubre de 2026

Sistema de préstamos descentralizado basado en la red blockchain de Ethereum
Desarrollado por: Aquilino Pascual Sanz
Dirigido por: Alberto Díaz Álvarez y Guillermo Sánchez Gutiérrez-Cabello
Proyecto Fin de Grado, 28 de octubre de 2026

E.T.S. de Ingeniería de Sistemas Informáticos
Campus Sur UPM, Carretera de Valencia (A-3), km 7
28031, Madrid, España

Si deseas citar este trabajo, la entrada completa en BibTeX es la siguiente:

```
@mastersthesis{citekey,
    title = {Sistema de préstamos descentralizado basado en la red blockchain
de Ethereum},
    type = {Bachelor's Thesis},
    author = {Pascual-Sanz, A.},
    school = {E.T.S. de Ingeniería de Sistemas Informáticos},
    year = {2026},
    month = {10},
}
```

Esta obra está bajo una licencia Creative Commons «Atribución-NoComercial-CompartirIgual 4.0 Internacional». Obra derivada de https://github.com/blazaid/UPM-Report-Template.



Todo cambio respecto a la obra original es responsabilidad exclusiva del presente autor.

Agradecimientos

A mis compañeros de universidad,
A los que me ayudaron, y a los que ayudé,
A los que me enseñaron, y a los que enseñé.
Lo mejor de toda la universidad.

Resumen

Este proyecto de fin de grado nace con el objetivo de desarrollar un sistema financiero descentralizado que ofrezca transparencia, seguridad y automatización en la gestión de préstamos. La motivación principal para realizar este proyecto radica en la necesidad de eliminar intermediarios innecesarios, reducir costes operativos y garantizar la trazabilidad de cada operación mediante contratos inteligentes en una red blockchain. Este enfoque brinda mayor confianza a los usuarios y permite una gestión eficiente de los préstamos mediante contratos ejecutados en la red, con una entidad prestamista responsable de la oferta y de los fondos.

Se ha desarrollado una solución basada en blockchain, contratos inteligentes y Web3 para descentralizar el proceso de préstamos, garantizando transparencia y eficiencia. Mediante contratos inteligentes, se automatiza la concesión, gestión y penalización por impago, reduciendo errores humanos. El sistema opera dentro de una dApp financiera, donde los préstamos se registran en la blockchain y se gestionan de forma segura mediante un contrato principal que administra los fondos, asegurando un servicio innovador, verificable y eficiente.

Palabras clave: blockchain, contrato inteligente, DeFi, Web3, Préstamos automatizados.

Abstract

This bachelor's thesis has been created with the aim of developing a decentralized financial system that offers transparency, security and automation in loan management. The main motivation for this project lies in the need to eliminate unnecessary intermediaries, reduce operating costs and ensure the traceability of each transaction through smart contracts on a blockchain network. This approach provides greater confidence to users and enables efficient loan management through on-chain contracts, while a single lending entity defines the products and supplies the funds.

A solution based on blockchain, smart contracts and Web3 has been developed to decentralize the loan process, guaranteeing transparency and efficiency. Through smart contracts, the granting, management and penalties for non-payment are automated, reducing human errors. The system operates within a financial decentralized application (dApp), where loans are recorded on the blockchain and securely managed through a master contract that manages the funds, ensuring an innovative, verifiable and efficient service.

Keywords: blockchain, smart contracts, DeFi, Web3, Automated loans.

Índice general

1 Introducción **1**

1.1 Objetivos 2

1.2 Motivación 3

1.3 Estructura de la memoria 4

2 Estado de la cuestión **6**

2.1 Protocolos de referencia en sistemas de préstamos descentralizados . . . 6

2.2 ¿Por qué blockchain y por qué Ethereum? 8

2.3 Tecnologías para el desarrollo 15

3 Arquitectura y desarrollo **19**

3.1 Solución propuesta 19

3.2 Limitaciones del proyecto 20

3.3 Análisis y diseño del software 22

3.4 Análisis de los contratos inteligentes implicados 32

3.5 Validación de resultados 45

4 Conclusiones **58**

4.1 Trabajo futuro 59

4.2 Impactos sociales y ambientales del desarrollo de este sistema a gran escala 60

A Código fuente de los contratos inteligentes **65**

A.1 PRESTAMOS_FACTORY.sol 65

B Código fuente de las pruebas **74**

B.1 test_prestamos_cursando.js 74

B.2 test_prestamos_factory.js 79

Índice de figuras

2.1 Diagrama simplificado del proceso de transacción en una blockchain. . . 11

2.2 Captura de información de mercado de Bitcoin, Ethereum, Tether, XRP, BNB y Solana. Fuente: CoinMarketCap [15]. 13

3.1 Diagrama de estados que asegura el control de los contratos del sistema. 20

3.2 Diagrama UML representando los actores del sistema relacionados con los casos de uso a los que tienen acceso. 23

3.3 Diagrama de secuencia que representa cómo la información vital del sistema queda registrada en la red aportando las características de inmutabilidad y confianza propias del proyecto. 24

3.4 Diagrama de entidad-relación. 32

3.5 Diagrama de entidad-relación que representa las relaciones y funciones entre los contratos inteligentes. 33

3.6 Diagrama de un ataque de reentrada 44

3.7 Ejemplo de inicio de sesión con MetaMask 46

3.8 Información de contratos previa a la comprobación mensual en el recuadro amarillo y balance de la entidad prestamista en el recuadro blanco. . 47

3.9 Información de contratos posterior a la comprobación mensual en el recuadro amarillo y balance de la entidad prestamista en el recuadro blanco. 48

3.10 Panel de control del usuario. 49

3.11 Información de Etherscan del registro en la red SepoliaETH del contrato inteligente principal 50

3.12 Registro en Sepolia en el que se puede ver una transacción o comprobante de pago Etherscan 51

4.1 Objetivos de desarrollo sostenible de la Agenda 2030 de las Naciones Unidas 63

Índice de tablas

3.1 Casos de uso del sistema 25

3.2 Casos de uso para desarrollos posteriores 26

3.3 Casos de uso 1 26

3.4 Casos de uso 2 26

3.5 Casos de uso 3 27

3.6 Casos de uso 4 27

3.7 Casos de uso 5 28

3.8 Casos de uso 6 28

3.9 Casos de uso 7 28

3.10 Casos de uso 8 29

3.11 Casos de uso 9 29

3.12 Casos de uso 10 29

3.13 Casos de uso 11 30

3.14 Definición de entidades del sistema 31

3.15 Descripción de atributos del Contrato PrestamosFactory 35

3.16 Descripción de funciones del Contrato PrestamosFactory 36

3.17 Descripción de atributos del Contrato PrestamoCursando 37

3.18 Descripción de funciones del Contrato PrestamoCursando 38

Índice de listados

3.1 Registro de préstamos en curso por usuario 33

3.2 Variables contrato préstamo 34

3.3 Función para pagar mensualidad 39

3.4 struct TipoContrato 39

3.5 Constructor de PrestamoCursando 40

3.6 Función que devuelve los contratos de un usuario 41

3.7 Recuperación de información de contratos 41

3.8 Muestra de información de contratos 42

3.9 Despliegue de contrato en la red local 53

3.10 Parametrización de préstamos en curso 53

A.1 Código fuente de PRESTAMOS_FACTORY.sol 65

B.1 Pruebas de préstamos en curso 74

B.2 Pruebas del contrato PRESTAMOS_FACTORY 79

1. Introducción

En un contexto económico global en el que algunos países enfrentan tasas de inflación elevadas, la adopción de monedas digitales como reserva de valor ha surgido como una alternativa atractiva para preservar el poder adquisitivo, intentando, de algún modo, evitar la erosión de valor que sufren las monedas locales.

Las criptomonedas ofrecen una opción descentralizada e internacional, proporcionando a los ciudadanos una alternativa que no está sujeta al control directo de los bancos centrales.

La existencia de criptomonedas no elimina por sí sola la dependencia de los bancos tradicionales y de otras entidades centralizadas para acceder a financiación. Este proyecto propone una alternativa basada en contratos inteligentes.

Hasta ahora, las necesidades de gestión y confianza necesarias para un sistema de préstamos solo las podían cubrir grandes entidades centralizadas, pero gracias a la tecnología blockchain y tal y como se explicará más adelante, estas características quedan a la disposición de cualquier ciudadano.

El presente Proyecto Fin de Grado aborda el diseño, desarrollo y validación de un prototipo de gestión de préstamos basado en la red Ethereum. La propuesta utiliza ETH como activo para la concesión y devolución de los préstamos, y contratos inteligentes para la gestión de estos.

Aunque los usuarios acceden al sistema mediante una aplicación descentralizada, la oferta de financiación queda centralizada en una única entidad, responsable de definir los productos y aportar los fondos.

La solución se organiza mediante un contrato inteligente principal que administra los productos y los fondos disponibles y que genera contratos inteligentes individuales que representan los préstamos contratados. En ellos se recoge la información necesaria para gestionar la deuda, las cuotas pendientes y las penalizaciones por impago. De este modo, la información del sistema que define la situación de los préstamos queda reflejada en la red Ethereum ofreciendo así las propiedades de veracidad e inmutabilidad necesarias para un sistema de este tipo.

Este proyecto analiza si un sistema de préstamos descentralizado basado en Ethereum puede ser accesible para los ciudadanos reduciendo la dependencia de financiación con modelos tradicionales.

El trabajo se centrará en la creación de un prototipo funcional capaz de comprobar la viabilidad técnica de esta propuesta haciendo uso de pruebas sobre los contratos inteligentes de Ethereum y validando los flujos de uso de la aplicación.

1.1. Objetivos

En esta sección se detallan los objetivos del proyecto. Se distinguen dos objetivos principales (Objetivo principal (OP)) y siete objetivos secundarios (Objetivo secundario (OS)), que concretan distintos aspectos de su desarrollo.

1.1.1. Objetivos principales

- **OP 1**: Desarrollo de un sistema de préstamos automatizado con registro y ejecución de operaciones en Ethereum, acceso directo mediante carteras y una entidad prestamista responsable de la oferta y de los fondos, cumpliendo los requisitos básicos de un proceso de préstamo, como pueden ser la seguridad, historial de cuotas, seguimiento, gestión de morosidad y personalización del usuario.
- **OP 2**: Desarrollar una interfaz unificada que permita a los usuarios, con o sin conocimientos técnicos, interactuar con el sistema de manera fácil e intuitiva. La interfaz garantizará que todas las funcionalidades sean accesibles desde un solo punto, facilitando la experiencia del usuario.

1.1.2. Objetivos secundarios

- **OS 1**: Explorar la viabilidad de los sistemas de identificación en aplicaciones DeFi.
- **OS 2**: Crear una solución segura y eficiente. Almacenar únicamente la información estrictamente necesaria para el funcionamiento correcto de la aplicación evitando así la exposición de datos sensibles y el procesamiento innecesario.

- **OS 3**: Optimizar la eficiencia de ejecución del código en la red blockchain para reducir la cantidad de gas necesario para el correcto funcionamiento del sistema.
- **OS 4**: Crear mecanismos de control de cumplimiento de contrato y penalización automatizados.
- **OS 5**: Explorar la flexibilidad y personalización de contratos inteligentes en productos financieros. Se investigarán maneras en las que el sistema sea capaz de ofrecer distintos productos financieros en función de la información que el sistema tenga del cliente como por ejemplo su situación financiera.
- **OS 6**: Optimizar el equilibrio entre la naturaleza pública y transparente de una red blockchain y la privacidad necesaria para los usuarios.
- **OS 7**: Analizar cómo se puede gestionar la liquidez en sistemas DeFi y cómo se puede coordinar la transferencia de fondos en un sistema de préstamos en una red blockchain.

1.2. Motivación

Demostrar la viabilidad de un sistema de préstamos descentralizado mostrando cómo la tecnología blockchain y los contratos inteligentes de Ethereum pueden ofrecer una alternativa sólida y segura frente a los sistemas financieros tradicionales, eliminando intermediarios y reduciendo costes y barreras de acceso.

Explorar y afrontar los desafíos técnicos específicos de desplegar un sistema DeFi en la red Ethereum.

Otra motivación es desarrollar un sistema seguro y eficiente en costes. Ambos aspectos son esenciales para su viabilidad económica y pueden convertirse en puntos débiles si no se abordan con suficiente detalle.

La motivación también radica en el valor social y económico que una solución como esta puede aportar a un mundo en el que la inclusión financiera sigue siendo una meta distante para millones de personas.

Enrique Dans, doctorado en sistemas de la información, explica en su blog que los proyectos DeFi responden a la creciente demanda de alternativas que no solo sean económicamente accesibles, sino también transparentes y seguras, proporcionando una

herramienta que permita a los usuarios mantener una mayor autonomía sobre sus decisiones financieras en un entorno digital global [1].

1.3. Estructura de la memoria

El código fuente de este proyecto se puede encontrar en el siguiente repositorio [2]. El código fuente con las pruebas unitarias de este proyecto se puede encontrar en el siguiente repositorio [3].

La estructura del documento es la siguiente:

- **Introducción:**
  - Presentación del problema y contexto económico de los préstamos descentralizados.
  - Motivación y relevancia del proyecto.
  - Objetivos generales y específicos del sistema.
- **Estado de la cuestión**:
  - Introducción a blockchain y su importancia en el sector financiero.
  - Explicación de Ethereum y su ecosistema.
  - Tecnologías clave (Web3, Solidity, Remix IDE, Node.js, Truffle, etc.).
- **Diseño de la Arquitectura:**
  - Solución propuesta: Cómo se estructura el sistema de préstamos descentralizado.
  - Suposiciones y limitaciones del proyecto.
  - Actores del sistema y su interacción (Usuarios, Propietarios, Contratos).
  - Casos de uso principales y pantallas del sistema.
  - Diagramas UML, diagramas de secuencia y diagramas entidad-relación.
- **Diseño detallado:**
  - Explicación a nivel técnico de cada entidad del sistema.

  - contratos inteligentes implicados y su funcionamiento.
  - Explicación detallada de la lógica del algoritmo principal.

- **Conclusiones y trabajo futuro**
  - Evaluación de los logros alcanzados en el proyecto.
  - Posibles mejoras y nuevas funcionalidades a explorar.

2. Estado de la cuestión

2.1. Protocolos de referencia en sistemas de préstamos descentralizados

El ecosistema de las finanzas descentralizadas (DeFi) ha experimentado un crecimiento significativo en los últimos años. Esto ha favorecido la aparición de distintos protocolos que ofrecen servicios financieros con características propias. El análisis siguiente se centra en sus aspectos tecnológicos y en su relación con la solución propuesta.

Cabe destacar que estos protocolos se han desarrollado en distintas redes blockchain y en el caso de Ethereum se puede acceder a los contratos inteligentes desplegados en la red, que son públicos y accesibles para cualquier usuario.

2.1.1. AAVE

El protocolo AAVE es el protocolo más parecido a la solución que ofrece este proyecto. La principal diferencia es que los usuarios finales no solo solicitan los préstamos, sino que también pueden ofrecer sus fondos para que otros usuarios puedan solicitar préstamos.

El proceso consiste en un fondo de liquidez. Este fondo de liquidez es alimentado por los usuarios que ofrecen sus fondos y estos pueden ser pedidos como préstamo por otros usuarios. La definición del tipo de préstamo y sus intereses viene definida por la garantía que puede aportar la persona que solicita el préstamo. Dependiendo del valor ofrecido como garantía se le asignará un tipo de interés y un límite de préstamo.

Se puede comprobar su estado en la red Ethereum en: *Estado del protocolo Aave V3 desplegado en la red* [4]

Para obtener más información se puede consultar la documentación oficial del protoco-

lo AAVE en: *Documentación del protocolo AAVE* [5]

2.1.2. Compound

Este protocolo es muy similar al anterior, la principal diferencia es que las garantías de devolución están previamente definidas según el tipo de préstamo que se solicita y no se permite la aportación de garantías externas.

Se puede comprobar su estado en la red Ethereum en: *Estado del protocolo Compound III desplegado en la red* [6]

Para obtener más información se puede consultar la documentación oficial del protocolo Compound en: *Documentación del protocolo Compound* [7]

2.1.3. Uniswap

Este protocolo es un intercambio descentralizado de criptomonedas, no es un protocolo orientado a préstamos aunque también se obtiene comisión a través de dejar prestados tus propios fondos en un fondo de liquidez. El funcionamiento consiste en aportar fondos a un fondo de liquidez. En este fondo de liquidez se definen dos criptomonedas de tal forma que los usuarios aprovechan la liquidez de esos activos que se encuentran en el fondo de liquidez para realizar intercambios de criptomonedas sin necesidad de un intermediario.

Por ejemplo, si un usuario quiere intercambiar ETH por Bitcoin, el protocolo Uniswap utiliza la liquidez del fondo de liquidez para realizar la transacción de manera automática y sin necesidad de conseguir un comprador que tenga exactamente Bitcoin y quiera exactamente ETH. En el contrato de intercambio se desglosa la comisión que obtiene la plataforma y el prestatario para la realización del intercambio.

Se puede comprobar su estado en la red Ethereum en: *Estado del protocolo Uniswap V4 desplegado en la red* [8]

Para obtener más información se puede consultar la documentación oficial del protocolo Uniswap en: *Documentación del protocolo Uniswap* [9]

A continuación se definirán las tecnologías utilizadas en la solución ofrecida por es-

te proyecto y también se definirán las claves por las que se han seleccionado dichas tecnologías, desde la decisión inicial de utilizar una red blockchain hasta pequeñas decisiones de diseño como la selección del entorno de trabajo para la UI.

2.2. ¿Por qué blockchain y por qué Ethereum?

2.2.1. blockchain

Las redes blockchain representan una revolución en el mundo digital ya que proporcionan una infraestructura segura, descentralizada y transparente para la transferencia y almacenamiento de datos en forma de transacciones.

Las blockchain son bases de datos distribuidas, esto quiere decir que la información se guarda en distintos nodos en forma de bloques donde cada uno tiene un hash que vincula cada bloque con el anterior en una cadena ordenada. Gracias a este sistema de bloques en cadena registrados por un hash se hace imposible la modificación de la información guardada en la red sin que deje una huella digital, ya que al realizar este acto se modificaría el hash asociado al bloque afectado, haciendo visible para el resto de copias distribuidas en la red la modificación, seguramente fraudulenta.

Esta estructura en cadena proporciona las propiedades de inmutabilidad y resistencia a manipulaciones.

Estos sistemas permiten la creación, registro y transferencia de información o valor sin necesidad de una entidad central de control, de aquí su naturaleza descentralizada.

Este diseño y estructura presentan características innovadoras que posicionan a las redes blockchain como una de las tecnologías más prometedoras, sobre todo en el sector financiero, aunque su aplicación se está extendiendo a diversas industrias tal y como se habla en el paper de la Universitat Politècnica de Catalunya [10].

Componentes en las transacciones blockchain

Para comprender en profundidad el funcionamiento y la seguridad de una red blockchain, es fundamental analizar sus componentes clave, su estructura de datos, los mecanismos de consenso para la aportación de nueva información a la red y las técnicas criptográficas que aseguran su integridad.

La siguiente información se ha obtenido a través del siguiente paper: *«INTRODUCCIÓN A LA TECNOLOGÍA BLOCKCHAIN: SU IMPACTO EN LAS CIENCIAS ECONÓMICAS»* [11]

1. **Bloques**: La red blockchain se estructura en bloques, que son conjuntos de información agrupada. Cada bloque contiene un conjunto de transacciones y un código de referencia al bloque anterior, denominado hash, formando una cadena continua. Los bloques también incluyen una marca de tiempo y un identificador único que garantiza su autenticidad.

2. **Hash criptográfico**: Cada bloque y cada transacción dentro de la blockchain están protegidos por un hash, que es un código único generado por los validadores. Este hash permite verificar de forma única el contenido del bloque y protege los datos de cualquier alteración. Si se modificara cualquier información en un bloque, el hash cambiaría, alertando al resto de la red de la alteración.

3. **Nodos**: Son las computadoras que participan en la red. Cada nodo contiene una copia completa o parcial de la blockchain y es responsable de verificar y propagar las nuevas transacciones y bloques. Los nodos se dividen en diferentes tipos:

   a) Nodos completos: Guardan la blockchain completa y participan en la verificación de bloques.

   b) Nodos ligeros: Descargan partes de la cadena y confían en nodos completos para la verificación.

   c) Nodos mineros: Son los encargados de replicar la blockchain, aceptando los bloques generados por los validadores y ejecutando las transacciones incluidas en los mismos. Estos nodos son los que inyectan transacciones en la red, a partir de fuentes externas al blockchain. Estos nodos compiten para agregar nuevos bloques a la cadena realizando el proceso de validación de transacciones y siendo incentivados por su labor.

4. **Proceso de validación de bloques**: Las redes utilizan mecanismos de consenso para acordar los bloques incorporados. A continuación se distinguen dos de ellos:

a) ***Proof of work*** **(Proof of Work, Prueba de trabajo (PoW))**: Forma de validación en la que los nodos mineros compiten por resolver un problema matemático que implica encontrar un hash que cumpla ciertos requisitos establecidos por el protocolo. El primer nodo en obtener este hash es el que se encarga de incorporar el nuevo bloque a la red y obtener la recompensa. Este proceso es difícil de resolver pero fácil de verificar para otros nodos.

b) ***Proof of Stake*** **(Proof of stake, Prueba de participación (PoS))**: En Ethereum, los validadores aportan ETH como garantía. El protocolo selecciona de forma pseudoaleatoria al proponente de cada bloque; otros validadores comprueban las transacciones y emiten votos sobre la cadena. Las recompensas incentivan la participación correcta y determinadas conductas deshonestas pueden provocar la pérdida de parte de los fondos depositados [12].

5. **Libro mayor distribuido**: La blockchain en conjunto actúa como un libro de contabilidad digital distribuido. Este libro mayor es inmutable, lo que significa que una vez registrada una transacción, no se puede modificar. Todos los nodos tienen acceso a la misma información, y las actualizaciones se sincronizan de forma continua y automática.

Proceso de transacción

Cada transacción en una blockchain sigue un proceso de verificación y registro antes de formar parte de un bloque:

1. **Creación de la transacción**: El usuario (o nodo emisor) crea una transacción con información como las direcciones del emisor y receptor, el monto de la transferencia y la firma digital generada con la clave privada del emisor.

2. **Verificación de la transacción**: Los nodos comprueban la firma y las condiciones de ejecución. El mecanismo de consenso de la red permite acordar los bloques que incorporan las transacciones válidas.

3. **Difusión a la red**: La transacción es difundida a la red para su verificación. Los nodos recopilan las transacciones válidas y las agrupan en bloques para su posterior validación.

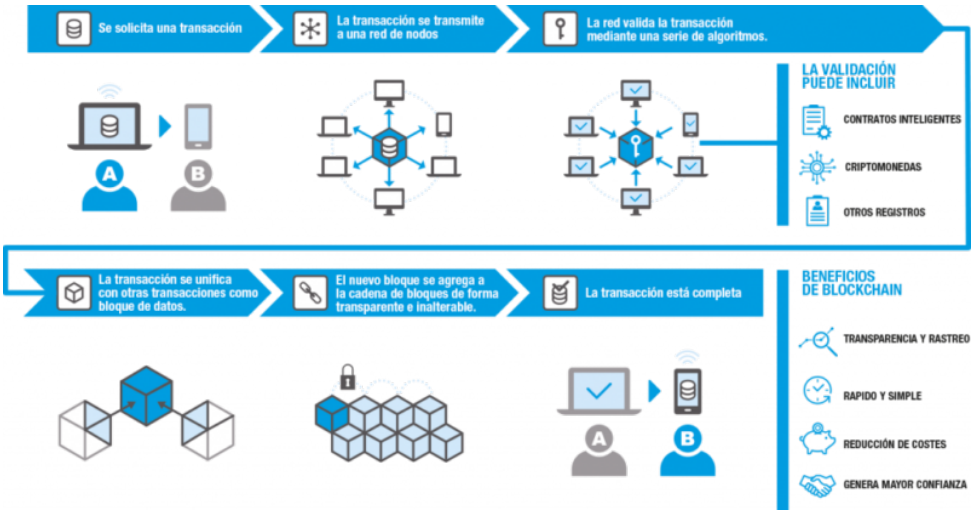
4. **Agregación a la blockchain**: El bloque verificado se agrega a la cadena. Este proceso implica registrar la transacción inicial y todas las demás del bloque y vincular

el nuevo bloque al bloque anterior mediante un hash. Esto asegura que la cadena sea inmutable y cronológica.

5. **Confirmación de la transacción**: Una vez que un bloque ha sido agregado, la transacción es confirmada. Las confirmaciones se refuerzan con cada bloque añadido posteriormente, haciendo la transacción prácticamente irreversible.

A través de la figura 2.1 se representa un diagrama simplificado del proceso de transacción.

Figura 2.1. Diagrama simplificado del proceso de transacción en una blockchain. Fuente: Neocheck [13]



Seguridad en blockchain

La seguridad de la blockchain depende de varias capas de tecnología criptográfica y del diseño descentralizado de la red. Los elementos clave de su seguridad incluyen:

1. **Inmutabilidad**: Una vez que se agrega un bloque, es extremadamente difícil modificar los datos sin alterar el hash que contiene sin afectar a todos los bloques posteriores. Cualquier intento de modificación es rápidamente detectado por los nodos validadores.

2. **Hashing criptográfico**: El uso de algoritmos de hashing (como SHA-256 en Bitcoin) convierte los datos de las transacciones en códigos únicos de longitud fija. Cualquier cambio en los datos originales generaría un hash completamente diferente, alertando a la red de la alteración. Este proceso es irreversible, lo que significa que el hash no puede descifrarse para revelar los datos originales.

3. **Distribución y descentralización**: La distribución del control entre numerosos nodos reduce la dependencia de un único participante. Las condiciones necesarias para mantener la seguridad dependen del protocolo de consenso de cada red [12].

4. **Criptografía de clave pública y privada**: Cada usuario de la red tiene una clave pública y una clave privada. La clave pública funciona como una dirección visible para recibir activos e identificarse en la red ante otros usuarios, mientras que la clave privada permite firmar transacciones y verificar la identidad del usuario, asegurando la autenticidad de la actividad en la red. Solo el propietario de la clave privada puede autorizar transacciones en nombre de su dirección pública.

Con todas estas características se puede afirmar que una red blockchain es una herramienta segura para la realización de transacciones monetarias y de reserva de valor. Ahora bien: ¿Cómo se puede crear un sistema automatizado? En redes como Bitcoin el único tipo de operación que se puede realizar es el de transacción. Este proyecto se apoya en una red con capacidad de ejecutar scripts o como son llamados dentro de la red de Ethereum: contratos inteligentes.

2.2.2. Ethereum

Historia

La información presentada a continuación sobre el origen de Ethereum se ha obtenido de su página web oficial [14].

Ethereum es una plataforma de blockchain propuesta en 2013 por Vitalik Buterin a sus 19 años de edad y fue lanzada oficialmente en 2015. Es una plataforma descentralizada gobernada por su comunidad y con el apoyo de la fundación Ethereum, sin ánimo de lucro.

Buterin mostró un talento matemático excepcional a edad temprana y comenzó a escribir artículos sobre Bitcoin, llegando a cofundar «Bitcoin Magazine 2012».

En 2014 publicó el «Documento técnico de Ethereum», en el que habla de la nueva plataforma que iría más allá del Bitcoin creando una cadena de bloques que podría hacer más que solo pagos.

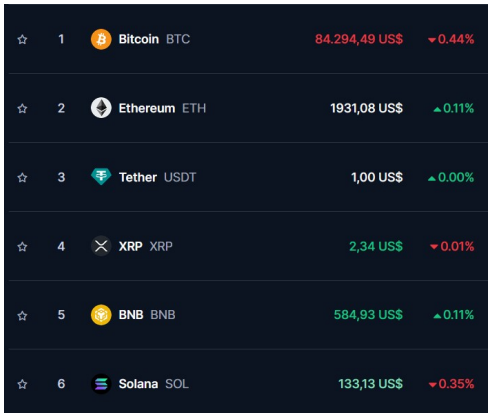
> "Ethereum amplía el enfoque de Bitcoin, básicamente diciendo: bueno, en lugar de tener reglas diseñadas para respaldar una sola aplicación, vamos a crear algo de propósito más general donde las personas puedan simplemente crear sus propias aplicaciones y las reglas para cualquier aplicación que hayan creado puedan ejecutarse e implementarse en la plataforma Ethereum." Ethereum.org [14]

2.2.3. Ethereum para el proyecto

El poderoso desarrollo de Vitalik es el motivo principal por el que se ha escogido la red de Ethereum: la capacidad de desplegar contratos inteligentes.

Ethereum se ha seleccionado por su capacidad para ejecutar contratos inteligentes y por su ecosistema de herramientas. La figura 2.2 muestra una captura de información de mercado de CoinMarketCap [15]. La imagen incluye, entre otros activos, ETH, BNB y SOL; no incorpora una fecha visible que permita confirmar el día de consulta.

Figura 2.2. Captura de información de mercado de Bitcoin, Ethereum, Tether, XRP, BNB y Solana. Fuente: CoinMarketCap [15].



Las características de Ethereum en las cuales se apoya este proyecto son:

1. **Contratos Inteligentes y Máquina Virtual de Ethereum (Ethereum Virtual Machine, Máquina Virtual de Ethereum (EVM))** Como se ha comentado antes, esta es la característica más importante de Ethereum. Los contratos inteligentes son programas autoejecutables con condiciones predefinidas, que permiten automatizar una amplia variedad de procesos y acuerdos entre partes sin necesidad de

intermediarios. La Máquina Virtual de Ethereum (EVM) es la infraestructura que hace posible la ejecución de estos contratos inteligentes. Esta consiste en un entorno virtual que permite que el código de los contratos inteligentes se ejecute por igual y de manera segura en todos los nodos de la red Ethereum, lo que garantiza consistencia e inmutabilidad. La EVM es compatible con varios lenguajes de programación, siendo Solidity el más utilizado para desarrollar contratos inteligentes en la red y el que se ha utilizado en este proyecto.

2. **Plataforma para dApps:** A partir del desarrollo de los contratos inteligentes, Ethereum fue la primera blockchain en introducir el concepto de aplicaciones descentralizadas (dApps). Las dApps son aplicaciones que funcionan en redes descentralizadas, permitiendo que los usuarios interactúen de manera directa y sin intermediarios en diferentes servicios, desde finanzas hasta juegos y redes sociales. En este caso, EVM actúa como una «App Store descentralizada»

3. **Criptoeconomía y Modelo de Tokenización:** La existencia de la criptomoneda ETH en Ethereum es clave para el desarrollo de un sistema DeFi, pudiendo ser utilizada para realizar transacciones y pagar comisiones obligatorias.

   Además, Ethereum introdujo el concepto de «tokenización», esto permite representar activos o derechos a través de Token. Para facilitar el uso y transferencia de estos, se han creado distintos estándares como por ejemplo:

   a) ERC-20 [16]: tokens fungibles, cuyas unidades son intercambiables entre sí, como puede ser una criptomoneda.

   b) ERC-721 [17] tokens no fungibles, que permiten distinguir cada token individualmente, pudiéndose usar para representar activos en la vida real o distintos derechos de usuario como una votación.

   c) ERC-1155 [18]: permite gestionar distintos tipos de tokens en una misma operación, tanto fungibles como no fungibles, en un mismo contrato.

4. **Identidad:** Ethereum es una red descentralizada, no utiliza identificaciones centralizadas como puede ser la centralización estatal del Documento Nacional de Identidad (DNI). Aunque no es una característica exclusiva de Ethereum, el sistema de identificación de esta red consiste en el típico sistema asimétrico de clave pública y privada de forma que para identificar a cada usuario se utiliza su clave pública y para que este pueda ejecutar transacciones en su nombre utiliza su clave única y privada. Sin embargo, esto también significa que no hay una forma nativa de verificar identidades del mundo real.

5. **Oráculos:** son servicios externos que alimentan la blockchain con datos de fuentes externas, permitiendo que los contratos inteligentes interactúen con el mundo real. Por ejemplo, en aplicaciones DeFi relacionadas con bienes raíces, un oráculo podría verificar que una propiedad física corresponde a un token en Ethereum, permitiendo así transacciones con respaldo en activos físicos.

Ethereum ofrece un entorno adecuado para desarrollar el proyecto gracias a la EVM y a la posibilidad de implementar contratos inteligentes. El prototipo utiliza ETH como activo de préstamo. La incorporación de garantías mediante NFT y datos externos se plantea como una posible ampliación futura.

2.3. Tecnologías para el desarrollo

2.3.1. Herramienta de identificación: MetaMask

La aplicación web necesita conectarse con la cuenta y dirección Ethereum del usuario para solicitar la firma de las operaciones de préstamo.

La primera opción podría ser desarrollar a través de herramientas de desarrollo Web3 una capa de identificación en la red, aunque se ha decidido delegar esta tarea a proyectos ya existentes y consolidados ya que es un punto crítico de seguridad del sistema y de esta manera se evita almacenar claves privadas de usuario.

Existen varias alternativas externas para identificación en blockchains en el sistema como Coinbase [19] y RabbyWallet [20] aunque simplemente se ha optado por MetaMask por su popularidad.

Estas herramientas de identificación proporcionan una API para comunicar la dApp con la cartera pudiendo automatizar a través de funciones Web3 las peticiones de autorización al usuario que ha entrado en el sistema.

En el caso de MetaMask, esta actúa como extensión del navegador y aparece como un Pop-up cada vez que la dApp requiere una autorización, también sirve para hacer transferencias directas entre cuentas, y por supuesto sirve de cartera.

2.3.2. Herramienta multiusos para despliegue en EVM Truffle

Para el desarrollo de una dApp en Ethereum Virtual Machine (EVM) es necesario realizar distintos procesos de compilación y despliegue para probar el funcionamiento del sistema antes de desplegar en la blockchain.

Truffle [21] es un conjunto de herramientas para el desarrollo de aplicaciones en EVM. Principalmente puede compilar contratos inteligentes y reducir el código a la información necesaria para la EVM.

Ganache [22] permite ejecutar una red local para realizar pruebas durante el desarrollo.

Red de pruebas Sepolia Aunque de manera muy técnica sea posible crear blockchain de manera local, debido a la importancia de probar todo desarrollo antes de desplegarlo en la red inmutable, existen alternativas de prueba de la mayoría de las redes blockchain públicas, estas actúan exactamente igual que la red original pero sin utilizar criptomonedas reales. Sepolia [23] es la red encargada de simular la red Ethereum, su moneda se denomina «SepoliaETH» y se puede obtener de manera gratuita en distintas plataformas.

2.3.3. Explorador de bloques Etherscan

La información guardada en la red está en distintas capas y de forma no legible para el humano. Etherscan [24] es un explorador de bloques de blockchain para Ethereum que permite visualizar transacciones, contrato inteligente, balances de carteras y más información en tiempo real. Es una herramienta clave para analizar y verificar actividades en la red Ethereum.

2.3.4. Remix IDE

Se ha de tener en cuenta que en el desarrollo de esta dApp aún no estaban tan extendidas las herramientas de desarrollo centralizado a través de inteligencia artificial, por lo que se encontró una alternativa muy buena para el desarrollo de contratos inteligentes con Solidity.

Remix Project [25] es el conjunto de herramientas más exhaustivo que puede usarse para toda la experiencia de desarrollo de contratos inteligentes. Ofrece todo tipo de herramientas para la depuración de código, despliegue en redes tanto públicas como privadas, compilación de contrato inteligente, etc.

2.3.5. Pruebas unitarias con Mocha

Marco de pruebas en JavaScript diseñado para ejecutar pruebas unitarias y de integración en aplicaciones Node.js. Ofrece la capacidad de comprobar reglas comunes a los contratos inteligentes como la comprobación de permisos de usuario, ingreso y retirada de fondos.

Tiene funciones para esperar operaciones asíncronas, muy importante a la hora de esperar acciones provenientes de la EVM, además de las funciones comunes a cualquier otro entorno de trabajo para pruebas.

2.3.6. Node.js

Node.js [26] es un entorno de ejecución JavaScript de código abierto y gratuito que facilita la incorporación de paquetes a un proyecto. Muy utilizado para instalar y utilizar librerías Web3.

2.3.7. Web3

El término Web3 se utiliza para describir aplicaciones y servicios web basados en tecnologías descentralizadas. En este proyecto también se utiliza la biblioteca Web3.js para conectar la aplicación con la red, consultar información e interactuar con los contratos inteligentes [27]. Los archivos de pruebas del anexo importan el paquete `web3`; esta copia del proyecto no incluye el archivo de dependencias necesario para identificar su versión exacta.

2.3.8. Solidity

Solidity [28] es un lenguaje de programación diseñado para escribir contratos inteligentes que se ejecutan en la EVM de Ethereum. Es similar a JavaScript y C++, con características específicas para gestionar transacciones, almacenamiento en blockchain y lógica descentralizada.

Solidity se ha utilizado por su integración con las herramientas del proyecto y por su orientación al desarrollo de contratos inteligentes para Ethereum.

2.3.9. Angular

Angular es un entorno de trabajo de código abierto basado en TypeScript. Sus herramientas hacen más sencillas las tareas comunes al desarrollo web como la gestión de rutas de dirección, organización de código a través de elementos llamados «componentes», recogida de datos de usuario, etc.

2.3.10. Ionic

Ionic es un entorno de trabajo de código abierto orientado al desarrollo de los componentes visuales en una aplicación web. En combinación con *Angular* se convierte en una herramienta que facilita mucho el desarrollo web.

3. Arquitectura y desarrollo

3.1. Solución propuesta

Este proyecto propone una dApp que gestiona préstamos de ETH dentro de la red Ethereum a través de contrato inteligente desplegados en dicha red.

Como se ha explicado en la comparación con otros protocolos DeFi, la diferenciación de este sistema es que solo una entidad, poseedora del proyecto, está autorizada a ofrecer los préstamos, configurar los tipos de préstamos disponibles y gestionar los fondos.

El sistema se basa en contratos inteligentes que registran en la red la información necesaria para gestionar los préstamos. Estos contratos permiten automatizar parte de la gestión y conservar un historial de las operaciones.

El usuario de la dApp interactúa con el sistema a través de una interfaz web en la que conectando su cartera con MetaMask puede gestionar sus propios préstamos sin la necesidad de interactuar directamente con la organización del sistema.

El seguimiento de control de pagos de los contratos funciona como una máquina de estados automatizada que es accionada como un semáforo con ciclos mensuales.

En la figura 3.1 se pueden ver los tres tipos de estados en los que puede encontrarse un préstamo al chequeo mensual de pagos (estado 0), cómo actúa la dApp dependiendo de su estado durante el chequeo (estado 1), y cómo queda el préstamo pasado el chequeo esperando al próximo (estado 0).

Figura 3.1. Diagrama de estados que asegura el control de los contratos del sistema.

ESTADO 0	ESTADO 1	ESTADO 0
	 **CHEQUEO DE MENSUALIDAD**	
PLAZO ABIERTO	PLAZO ABIERTO (PENALIZACIÓN)	PLAZO ABIERTO
MES PAGADO	MES PAGADO (CONTINUA CON NORMALIDAD)	PLAZO ABIERTO
FINALIZADO	FINALIZADO (SE TANSFIEREN FONDOS A CONTRACT FACTORY Y SE BORRA DE MAPPING DE PRÉSTAMOS ACTIVOS)	FINALIZADO (ESPERANDO BORRADO)

3.2. Limitaciones del proyecto

3.2.1. Limitaciones legales

El Reglamento (UE) 2023/1114, de 31 de mayo de 2023, relativo a los mercados de criptoactivos (MiCA), entró en vigor el 29 de junio de 2023. Su aplicación general comenzó el 30 de diciembre de 2024; los títulos III y IV se aplican desde el 30 de junio de 2024, con las excepciones y disposiciones transitorias previstas en el propio reglamento [31].

La aplicación de MiCA depende de las actividades y de los sujetos implicados. El considerando 22 distingue los servicios prestados de forma plenamente descentralizada sin intermediarios. La existencia de una entidad que administra la oferta y los fondos del prototipo obliga a estudiar el modelo concreto antes de extraer conclusiones sobre su encaje normativo.

Los importes del artículo 35 corresponden a emisores de fichas referenciadas a activos: sus fondos propios deben alcanzar, como mínimo, el mayor de 350 000 €, el 2 % del importe medio de los activos de reserva o la cuarta parte de los gastos fijos generales del año anterior. Los proveedores de servicios de criptoactivos tienen un régimen prudencial distinto en el artículo 67. Estas cifras no constituyen un requisito general para cualquier aplicación de préstamos en ETH [31].

Por ello, una explotación real requeriría determinar las actividades reguladas, las autorizaciones exigibles y las obligaciones aplicables a la entidad prestamista. Este prototipo no emite una ficha referenciada a activos ni permite deducir por sí solo que sean exigibles los umbrales del artículo 35.

3.2.2. Limitaciones técnicas

El sistema desarrollado presenta una serie de limitaciones técnicas derivadas tanto del alcance definido para el proyecto como de las características propias de una solución basada en contratos inteligentes. Las principales limitaciones identificadas son las siguientes:

1. **Pagos anticipados:** el sistema no contempla actualmente la realización de pagos anticipados sobre los préstamos contratados.
2. **Cancelación anticipada:** no se contempla la posibilidad de cancelar un préstamo antes de que finalice el periodo establecido inicialmente en el momento de su contratación.
3. **Ausencia de garantías o colateral:** el sistema no incorpora mecanismos de colateralización que permitan garantizar el pago del préstamo en caso de impago.
4. **Uso de un único activo:** el prototipo utiliza exclusivamente ETH como activo para la concesión y devolución de los préstamos. No se contempla actualmente el uso de otros tokens o activos digitales como unidad de préstamo.

5. **Escalabilidad y consumo de gas:** el incremento del número de préstamos activos implica un mayor número de operaciones que deben ser procesadas y registradas en la blockchain. En consecuencia, el coste acumulado de gas puede aumentar a medida que crece el volumen de préstamos gestionados. Esta situación no tiene por qué invalidar el proyecto, pero sí exige un desarrollo orientado a mejorar su escalabilidad.

Estas limitaciones deben tenerse en cuenta al valorar una posible aplicación del sistema en un entorno real a la vez que se plantean como líneas de desarrollo futuro.

3.3. Análisis y diseño del software

En esta sección se muestran diagramas y tablas que definen la estructura y el flujo del sistema.

3.3.1. Actores del sistema

Los actores que pueden participar en el sistema son:

- **Usuario prestatario (User):** Usuario que accede al servicio de préstamos.
- **Entidad prestamista (Owner):** Entidad que ofrece los distintos tipos de préstamos.

Diagrama UML

En la figura 3.2 hay representado un Diagrama-UML que relaciona los actores del sistema con los «Casos de Uso» que pueden realizar.

Figura 3.2. Diagrama UML representando los actores del sistema relacionados con los casos de uso a los que tienen acceso.

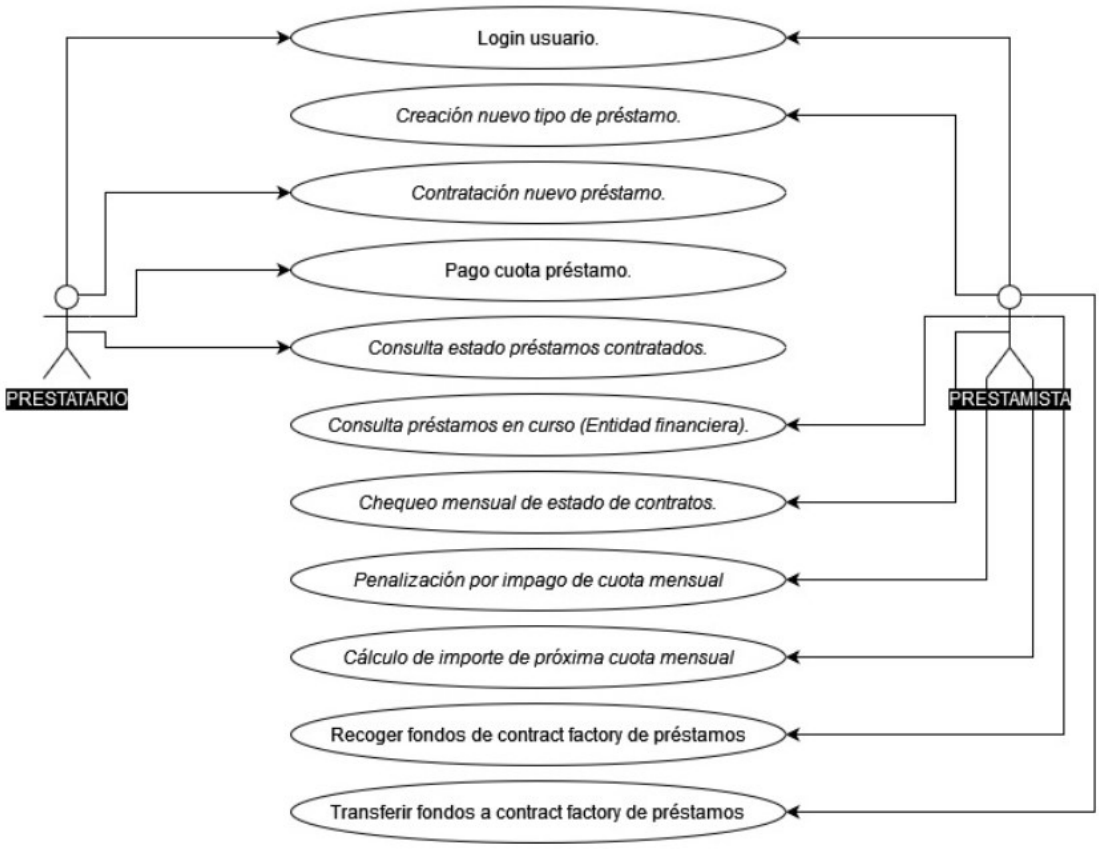
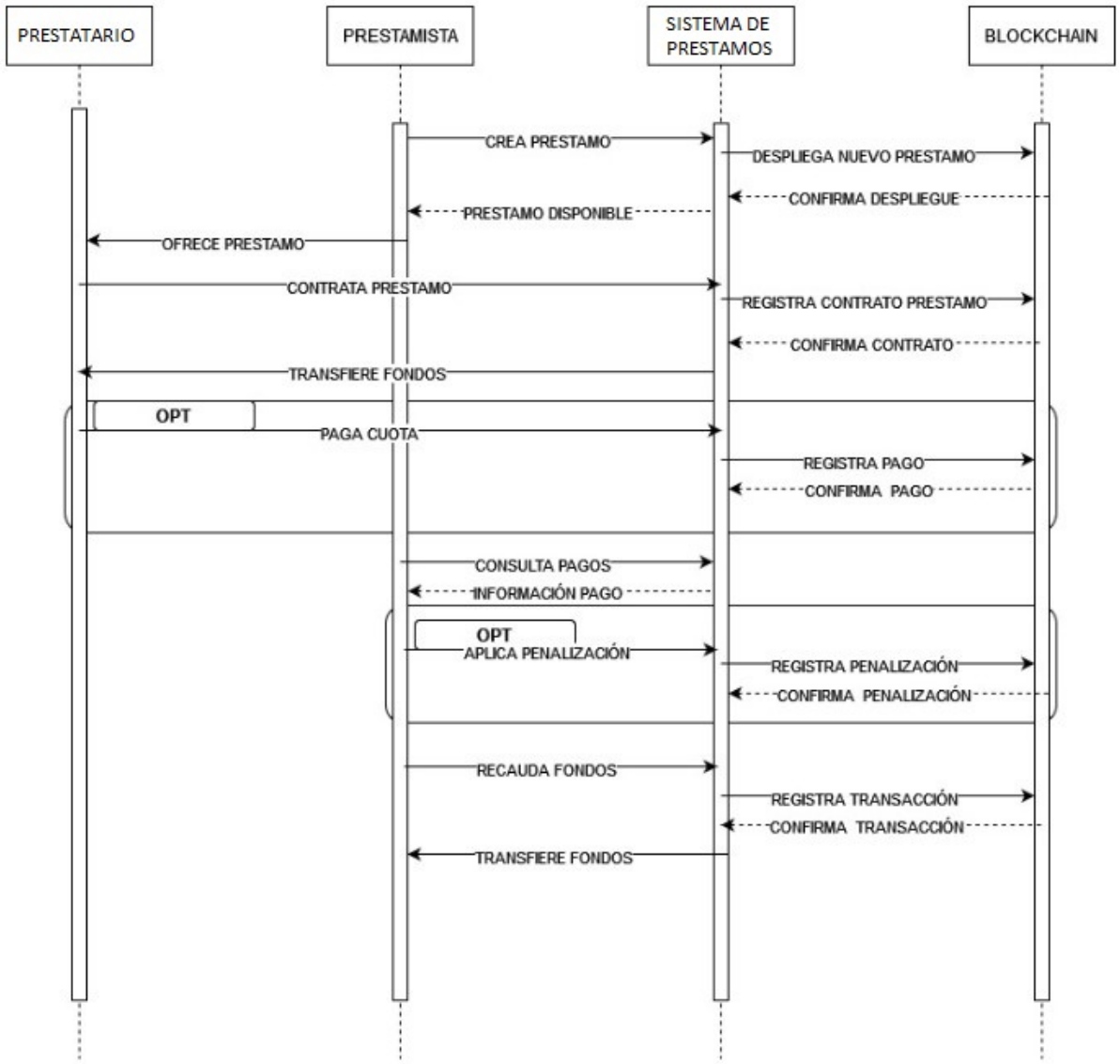


Diagrama de secuencia UML

Para mostrar cómo interactúan los actores con el sistema en una línea de tiempo y conocer el flujo de información que sucede en el sistema se ha definido el siguiente Diagrama de secuencia UML en la figura 3.3.

En este caso, el diagrama de secuencia muestra cómo todo registro en el sistema siempre es grabado en la blockchain, siendo un flujo clave en este proyecto para poder dar las propiedades de veracidad e inmutabilidad a la información de los préstamos.

Figura 3.3. Diagrama de secuencia que representa cómo la información vital del sistema queda registrada en la red aportando las características de inmutabilidad y confianza propias del proyecto.



3.3.2. Casos de uso

En el marco del desarrollo del presente proyecto, se ha priorizado la definición de un conjunto de casos de uso que representan los escenarios esenciales para cumplir con los objetivos fundamentales del sistema, de acuerdo con el alcance limitado del proyecto.

La selección de los casos de uso no pretende abarcar una totalidad de posibles funciona-

lidades del sistema sino un conjunto operativo sólido y funcional, comentando también un posible desarrollo futuro.

Estos casos de uso se definen en las tablas 3.1 y 3.2.

Pantallas del sistema

Para facilitar la comprensión de los casos de uso, se enumeran las principales pantallas del sistema:

- **Pantalla principal común:** Pantalla de inicio común a todos los usuarios, principalmente para identificarse en el sistema.
- **Pantalla principal del prestamista.**
- **Pantalla principal del prestatario.**

Casos de uso

Tabla 3.1. Casos de uso del sistema

Código	Descripción	Prioridad	Dependencia
CU1	Inicio de sesión usuario	10	
CU2	Creación nuevo tipo de préstamo	9	
CU3	Contratación nuevo préstamo	8	CU2
CU4	Consulta estado préstamos contratados	7	CU3
CU5	Pago cuota préstamo	6	CU3
CU6	Consulta préstamos en curso (Entidad prestamista)	5	CU3
CU7	Chequeo mensual de estado de contratos	4	
CU8	Penalización por impago de cuota mensual	3	CU7
CU9	Cálculo de importe de próxima cuota mensual	2	CU3
CU10	Retirar fondos del sistema	1	
CU11	Ingresar fondos en el sistema	1	

Casos de uso para posteriores desarrollos

Tabla 3.2. Casos de uso para desarrollos posteriores

Código	Descripción
CU12	Garantía de devolución en caso de impago
CU13	Consulta del valor del Euribor
CU14	Validación de solvencia económica de usuarios

Casos de uso detallados

En esta sección se describen los casos de uso implementados y más adelante, en la sección de trabajo futuro se comentarán los casos de uso futuros.

Tabla 3.3. Casos de uso 1

CU1	inicio de sesión del usuario.
Descripción	Los usuarios se identificarán en el sistema a través de MetaMask.
Pantalla	Pantalla inicial. (Se redirecciona a esta página si no se ha realizado el inicio de sesión).
Rol	Cualquier usuario.
Flujo	- El usuario, al acceder a la página web, recibirá una notificación por parte de MetaMask para poder acceder al servicio. - La extensión de MetaMask permite la autenticación del usuario y la página se redirige a la página correspondiente a su rol.

Tabla 3.4. Casos de uso 2

CU2	Creación nuevo tipo de préstamo.
Descripción	La entidad prestamista puede crear nuevos productos tipo «préstamo».
Pantalla	Pantalla principal de la entidad prestamista.
Rol	Owner.
Flujo	- El usuario owner tiene un formulario en su página principal para rellenar la información necesaria para la creación de un nuevo tipo de préstamo. - Este contrato se le mostrará al usuario prestatario en su página principal en forma de producto para contratar.

Tabla 3.5. Casos de uso 3

CU3	Contratación nuevo préstamo.
Descripción	Los usuarios prestatarios pueden seleccionar el préstamo que desean contratar.
Pantalla	Pantalla principal de usuarios prestatarios.
Rol	User.
Flujo	- En la página principal el usuario podrá visualizar los posibles préstamos que puede contratar. - El usuario contrata un nuevo préstamo pinchando en el botón de «Contratar». Tan pronto como se procese la transacción, la dApp llamará al contrato inteligente desplegado en la red el cual genera otro contrato inteligente que refleja el estado del préstamo en cada momento. Al mismo tiempo, las unidades de ETH solicitadas en el préstamo serán transferidas automáticamente desde los fondos del contrato inteligente que conserva el balance del sistema a la dirección del usuario.

Tabla 3.6. Casos de uso 4

CU4	Consulta estado préstamos contratados.
Descripción	Los usuarios del servicio podrán consultar el estado de sus préstamos contratados visualizándolos en su página principal.
Pantalla	Pantalla principal usuario.
Rol	User.
Flujo	En el momento en que el usuario hace inicio de sesión ya puede visualizar los contratos que tiene contratados.

Tabla 3.7. Casos de uso 5

CU5	Pago cuota préstamo.
Descripción	Los usuarios del servicio de préstamo realizan el pago de sus mensualidades a través de la dApp del proyecto.
Pantalla	Pantalla principal de usuarios prestatarios.
Rol	User.
Flujo	- En la página principal del usuario aparecen los distintos préstamos que tiene contratados y sus respectivos estados, junto a esto, existe un botón para cada contrato que transfiere la mensualidad calculada previamente desde la cuenta del usuario a la dirección del contrato inteligente de préstamo.

Tabla 3.8. Casos de uso 6

CU6	Consulta préstamos en curso (Entidad prestamista).
Descripción	La entidad prestamista puede consultar el estado de cada préstamo en curso.
Pantalla	Pantalla principal de la entidad prestamista.
Rol	Owner.
Flujo	- En la página principal del propietario aparecen los contratos en curso.

Tabla 3.9. Casos de uso 7

CU7	Chequeo mensual de estado de contratos.
Descripción	La entidad prestamista ejecuta la orden de comprobar qué mensualidades no han sido pagadas ese mes.
Pantalla	Pantalla principal de entidad prestamista.
Rol	Owner.
Flujo	Desde la página principal del propietario se tiene acceso al botón que lanza esa orden siempre y cuando hayan pasado 30 días desde la última orden.

Tabla 3.10. Casos de uso 8

CU8	Penalización por impago de cuota mensual.
Descripción	Los contratos en estado de impago generan una penalización de deuda, esta penalización solo puede suceder una vez al mes.
Pantalla	Pantalla principal de entidad prestamista.
Rol	Owner.
Flujo	Cuando el propietario genera la orden de chequeo mensual los contratos que tengan estado de «impago» aumentarán su valor de deuda.

Tabla 3.11. Casos de uso 9

CU9	Cálculo de importe de próxima cuota mensual.
Descripción	El importe de la próxima cuota mensual es un valor dinámico sujeto a penalizaciones.
Pantalla	Pantalla principal usuario.
Rol	User.
Flujo	- Tras el chequeo mensual la cuota del próximo pago se podrá ver modificada en caso de penalización. - En la información de los préstamos contratados aparecerá la cantidad de la cuota del mes en curso.

Tabla 3.12. Casos de uso 10

CU10	Retirar fondos del sistema.
Descripción	La entidad prestamista propietaria del sistema de préstamos puede disponer de los fondos almacenados en el sistema cuando desee.
Pantalla	Pantalla principal de Owner.
Rol	Owner.
Flujo	La entidad prestamista tendrá una casilla en su página principal para escribir y recibir el importe desde la dirección del contrato inteligente con el balance a su dirección de la red.

Tabla 3.13. Casos de uso 11

CU11	Ingresar fondos en el sistema.
Descripción	La entidad prestamista puede transferir cualquier cantidad de ETH al sistema de préstamos para mantener un saldo positivo y poder realizar estos préstamos.
Pantalla	Pantalla principal Owner.
Rol	Owner.
Flujo	La entidad tendrá una casilla en su página principal para escribir y transferir el importe deseado a la dirección del sistema de préstamos.

3.3.3. Entidades del sistema

A continuación se definen las entidades cuyas relaciones compondrán y habilitarán las funciones del sistema propuesto.

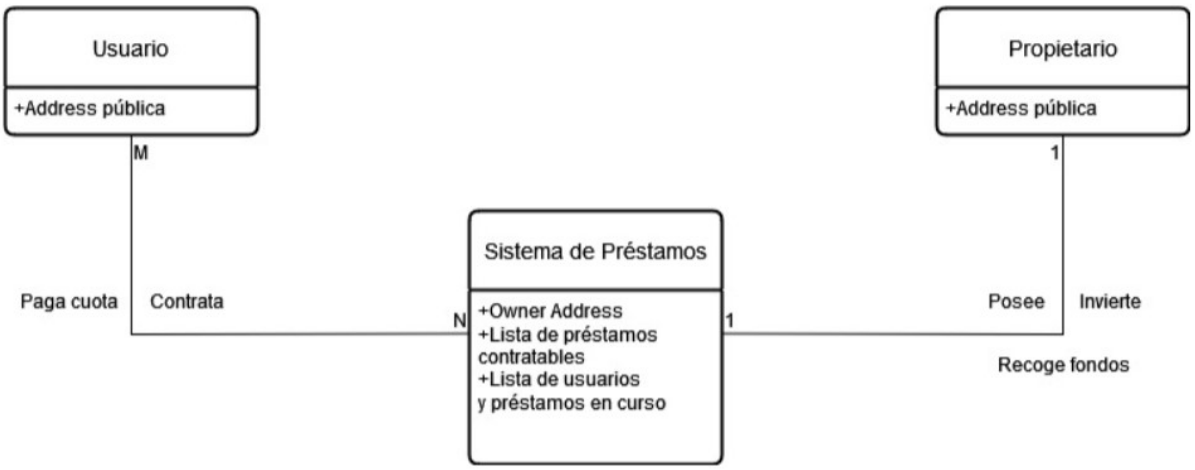
Tabla 3.14. Definición de entidades del sistema

1	Usuario.
Descripción	Usuario que accede al servicio de préstamos.
Atributos	Dirección pública de la red.
Acciones	- Pedir préstamo. - Pagar préstamo.
2	Propietario.
Descripción	Propietario del sistema, único capaz de recolectar los fondos del sistema. Normalmente una entidad prestamista.
Atributos	Dirección pública de la red.
Acciones	- Transferir unidades al sistema. - Recoger unidades del sistema.
3	Sistema de préstamos.
Descripción	Conjunto de contratos inteligentes que definen los elementos del sistema y cuya ejecución valida las transacciones del sistema.
Atributos	- Dirección de propietario. - Lista de préstamos contratables. - Lista de usuarios junto con sus préstamos en curso.
Acciones	- Ejecutar las transacciones ordenadas por los usuarios del sistema. - Registrar en la red Ethereum la información del sistema.

Diagrama de entidad-relación

El diagrama representado en la figura 3.4 pretende definir mediante relaciones entre las entidades las funciones que definen el sistema.

Figura 3.4. Diagrama de entidad-relación.



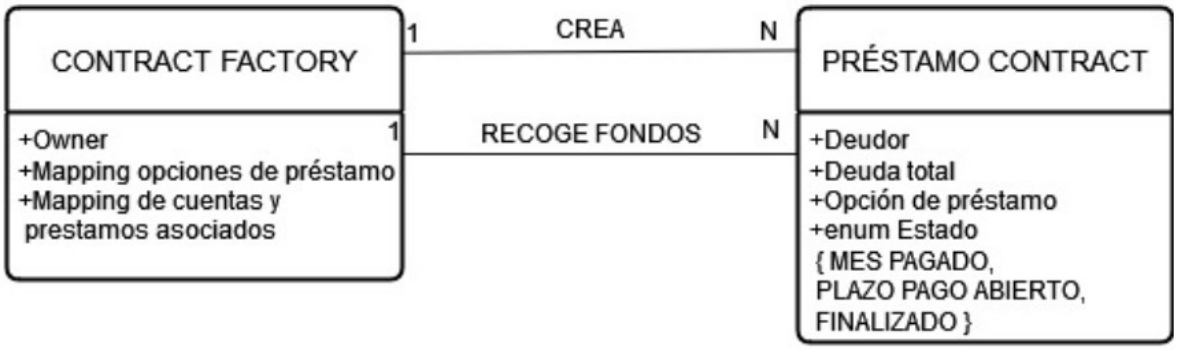
3.4. Análisis de los contratos inteligentes implicados

En esta sección se define la arquitectura de diseño utilizada para los contratos inteligentes, así como los atributos (tablas 3.15 y 3.17) y funciones (tablas 3.16 y 3.18) de cada uno de ellos.

3.4.1. Diagrama de entidad-relación de contratos inteligentes

Aunque no sean entidades del sistema, para explicar detalladamente la arquitectura de diseño se define otro diagrama de entidad-relación en la figura 3.5.

Figura 3.5. Diagrama de entidad-relación que representa las relaciones y funciones entre los contratos inteligentes.



PrestamosFactory

La figura 3.5 también define gráficamente el patrón de diseño utilizado en el desarrollo de los contratos inteligentes. A este patrón se le denomina «Factory».

El diseño de «Factory» encaja perfectamente en el desarrollo de este proyecto ya que nos permite crear los distintos contratos de préstamo de forma dinámica y automática. En esta arquitectura, un contrato principal (el *Factory*) actúa como intermediario recibiendo las solicitudes y generando nuevos contratos (*contratos préstamo*) según las necesidades de los usuarios o la aplicación. Dentro de este contrato se definen los distintos tipos de préstamos que pueden ser contratados por los usuarios, así como las distintas funcionalidades de gestión y monitorización. Para ello este contrato cuenta con variables como las del listado siguiente:

Listado 3.1. Registro de préstamos en curso por usuario

```
1    mapping(address => PrestamoCursando[])  prestamosCursando;
2    address[] clientes;
```

En el fragmento de código del listado 3.1 se observa la variable *prestamosCursando* perteneciente al contrato inteligente principal *PrestamosFactory*, la cual guarda en la red blockchain el conjunto de direcciones correspondientes a un contrato tipo *Préstamo* pertenecientes al sistema.

Por otra parte, cada contrato creado por *PrestamosFactory*, es decir, cada contrato de préstamo, mantiene información específica como la dirección del prestatario, la deuda restante, el número de cuotas pendientes, el estado del préstamo o el porcentaje de penalización aplicable, tal y como se muestra en el listado 3.2.

Listado 3.2. Variables contrato préstamo

```
1            uint index;
2            enum State { PAGADO, ABIERTO, FINALIZADO }
3            State state;
4            address prestatario;
5            address entidadFinanciera;
6            address payable prestamosFactory;
7            uint256 cuotasRestantes;
8            uint256 cantidadPrestada;
9            uint256 deudaRestante;
10           uint256 cuotaMensual;
11           uint256 penalizacion;
12           uint256 ultimoCheckeo;
13
```

3.4.2. Análisis de contratos inteligentes

PrestamosFactory

El contrato *PrestamosFactory* centraliza la gestión de los tipos de préstamo disponibles y el registro de los préstamos asociados a cada usuario. Para ello, mantiene un conjunto de atributos que identifican al propietario del sistema, definen las condiciones de los productos y relacionan a los usuarios con sus contratos de préstamo. La tabla 3.15 recoge estos atributos.

Tabla 3.15. Descripción de atributos del Contrato PrestamosFactory

Nombre	Tipo de dato	Descripción
Owner	Tipo Address	Dirección de la red Ethereum perteneciente a la propietaria del sistema.
Tipos de préstamo	`TipoContrato[]`	Array que guarda los tipos de préstamo contratables por el usuario. Esta variable solo puede ser editada si el usuario se identifica con la dirección del propietario.
TipoPréstamo	Struct	Conjunto de variables que definen un tipo de préstamo (importe, intereses, deuda total, penalización de impago y cuotas)
Préstamos en curso	`mapping(address => Prestamo Cursando[])`	Mapping que relaciona la dirección de un usuario con las direcciones de sus préstamos en curso. El contrato principal actualiza este registro al contratar o eliminar un préstamo.

A continuación, la tabla 3.16 presenta las funciones del contrato *PrestamosFactory*, sus parámetros de entrada y salida y una descripción de su finalidad.

Tabla 3.16. Descripción de funciones del Contrato PrestamosFactory

Nombre	Entrada	Salida	Descripción
`definirPrestamo`	Interés, importe, cuotas y penalización	—	Crea un tipo de préstamo y emite el evento correspondiente. Solo puede ejecutarla el propietario.
`contratar Prestamo`	Prestatario e índice del tipo	—	Crea un contrato individual, lo registra y transfiere el importe al prestatario.
`getClientes`	—	Direcciones	Devuelve las direcciones de los clientes registrados. Solo puede consultarlas el propietario.
`recolectar Fondos`	Importe	—	Transfiere fondos del contrato principal al propietario.
`invertir`	Valor de la transacción	—	Recibe fondos en el contrato principal y emite un evento.
`verContratos`	Dirección del prestatario	Contratos de préstamo	Devuelve los contratos asociados al usuario. Solo pueden consultarlos el propietario o el prestatario.
`verTipos Contrato`	—	Tipos de préstamo	Devuelve los productos disponibles.
`getBalance`	—	Saldo	Devuelve el saldo del contrato principal al propietario.
`eliminar Prestamo Finalizado`	Usuario e índice	—	Elimina del registro un préstamo finalizado y actualiza los índices.

PrestamoCursando

El contrato inteligente `PrestamoCursando` representa un préstamo individual contratado por un usuario. Almacena las condiciones del préstamo, permite el pago de las

cuotas y mantiene actualizado su estado tras el chequeo de mensualidad. A continuación, en la tabla 3.17 se describen los atributos implicados.

Tabla 3.17. Descripción de atributos del Contrato PrestamoCursando

Nombre	Tipo de dato	Descripción
`index`	`uint`	Posición del préstamo en el registro del usuario.
`state`	`State`	Estado: `PAGADO`, `ABIERTO` o `FINALIZADO`.
`cuotas Restantes`	`uint256`	Número de mensualidades pendientes.
`prestatario`	`address`	Dirección de la cuenta que contrata el préstamo.
`deudaRestante`	`uint256`	Deuda pendiente, expresada en wei.
`cuotaMensual`	`uint256`	Importe de la próxima cuota, expresado en wei.
`penalizacion`	`uint256`	Porcentaje aplicado a la deuda en caso de impago.

El encargado de las operaciones de actualización de importes del *contrato de préstamo* es el propio contrato inteligente a través de las funciones explicadas en la tabla 3.18.

Tabla 3.18. Descripción de funciones del Contrato PrestamoCursando

Nombre	Entrada	Salida	Descripción
`pagarMensualidad`	Valor de la cuota	—	Comprueba el pago, actualiza deuda y estado y envía los fondos al contrato principal.
`penalizarPrestamo`	—	—	Función privada que aumenta la deuda por impago y recalcula la cuota.
`calculoCuota Mensual`	—	Cuota en wei	Función privada que calcula la cuota; incorpora el resto de la división a la última mensualidad.
`checkeoMensual`	—	—	Actualiza el estado y aplica la penalización o elimina el préstamo finalizado. Exige el plazo y la cuenta de la entidad prestamista.

3.4.3. Decisiones de implementación en el sistema

Durante el desarrollo de los contratos inteligentes se han tomado decisiones técnicas sobre la distribución de responsabilidades entre la aplicación y la red, el control de acceso y el registro de las transacciones. A continuación, se presentan las principales decisiones adoptadas.

Controles de acceso reflejados en la propia red: Al contar cada *contrato de préstamo* con su propia información, se garantiza que cada préstamo es independiente y el sistema de control de acceso puede ser gestionado de manera casi autónoma si está reflejado en la blockchain.

Se puede ver un ejemplo de gestión autónoma en la función del listado 3.3 del *contrato de préstamo*, en la que a través de las líneas que comienzan por `require`, aseguramos que:

- El prestatario sea el único que puede realizar el pago de la mensualidad.

- El importe enviado es exactamente el valor de la cuota mensual
- El estado del préstamo permite realizar el pago.

Listado 3.3. Función para pagar mensualidad

```
1            function pagarMensualidad() payable public {
2                require(msg.sender == prestatario, "Only borrower");
3                require(msg.value == cuotaMensual, "Send exact debt value");
4                require(state == State.ABIERTO, "Payment period not open");
5
6                state = State.PAGADO;
7                deudaRestante -= cuotaMensual;
8                cuotasRestantes--;
9
10               if(cuotasRestantes == 1){   cuotaMensual =
  calculoCuotaMensual(); }
11
12               if(cuotasRestantes == 0){  state = State.FINALIZADO;  }
13               transferirFondosAFactory(msg.value);  }
14
```

Contrato préstamo *abstracto* Para que la creación de varios tipos de *contrato de préstamo* sea dinámica y se puedan definir desde la dApp se ha optado por crear un único contrato que pueda ser parametrizado en el momento de su creación. En el contrato *factory* se define una estructura, mostrada en el listado 3.4, que contiene los parámetros de cada tipo de préstamo, como el importe, los intereses, la penalización por impago y el número de cuotas.

Listado 3.4. struct TipoContrato

```
1                struct TipoContrato {
2                    uint256 index;
3                    uint256 porcentajeInteres;
4                    uint256 cantidad;
5                    uint256 deudaTotal;
6                    uint256 cuotas;
7                    uint256 penalizacionImpago;
8                }
```

```
9
```

Cuando un usuario solicita un préstamo, se pasa el tipo de préstamo seleccionado al constructor del contrato de préstamo, mostrado en el listado 3.5, que inicializa sus variables con los valores correspondientes.

Listado 3.5. Constructor de PrestamoCursando

```
1               constructor(uint256 _index, address _prestatario, address
  _entidad, uint256 _cuotasRestantes, uint256 _cantidad, uint256 _deudaTotal
  , uint256 _penalizacion){
2                   index = _index;
3                   state = State.ABIERTO;
4                   prestatario = _prestatario;
5                   entidadFinanciera = _entidad;
6                   prestamosFactory = payable(msg.sender);
7                   cuotasRestantes = _cuotasRestantes;
8                   cantidadPrestada = _cantidad;
9                   deudaRestante = _deudaTotal;
10                  penalizacion = _penalizacion;
11                  cuotaMensual = calculoCuotaMensual();
12                  ultimoCheckeo = block.timestamp;
13              }
14
```

Direcciones como identificación

En un sistema basado en Ethereum no es necesario generar un identificador centralizado para identificar a un usuario. La dirección pública de su cartera constituye un identificador único dentro de la red. Cada solicitud de préstamo genera un nuevo contrato con una dirección propia. El contrato principal registra su asociación con el prestatario en la variable `prestamosCursando`. De este modo, se pueden consultar y administrar los préstamos vinculados a cada usuario mediante la función del listado 3.6.

Listado 3.6. Función que devuelve los contratos de un usuario

```
1     function verContratos(address _prestatario) public view returns (
      PrestamoCursando[] memory){
2         require((msg.sender == owner)||(msg.sender == _prestatario), "Top
      Secret");
3         return (prestamosCursando[_prestatario]);
4     }
```

Representación de cantidades monetarias

Las cantidades monetarias se almacenan como enteros sin signo de tipo `uint256`, expresados en wei. La división entre enteros en Solidity trunca el resultado hacia cero [32]. Por ello, el cálculo de una cuota puede dejar un resto; el contrato incorpora ese resto a la última cuota. Esta decisión responde al uso de aritmética entera y a la unidad mínima del activo, no a un fallo de la versión del lenguaje.

Reducción del consumo de gas

Siempre que una operación no requiere necesariamente modificar el estado de la blockchain, esta se ejecuta en la aplicación web del cliente. De esta manera se evita utilizar gas para operaciones puramente relacionadas con la visualización de información, como el recorrido de los contratos de un usuario para mostrar su estado. En este sistema se recupera la información de todos los contratos 3.7 y es la aplicación web la que filtra y muestra la información de cada contrato (fragmento de código 3.8). El primer fragmento de código recupera la información de la red y el segundo la muestra por pantalla al usuario.

Listado 3.7. Recuperación de información de contratos

```
1         async iniciarVistaContratados(){
2     this.direccionesContratados = await this.factoryContract.methods
3         .verContratos(localStorage.getItem('userAddress'))
4             .call({ from: localStorage.getItem('userAddress')});
5
6         this.direccionesContratados.forEach(async (direccion: any) => {
```

```
7         var prestamoCursando = new this.web3.eth.Contract(PRESTAMOCURSANDO.
  abi, direccion);
8         var contratado = await prestamoCursando.methods.mostrarInfo().call({
  from: localStorage.getItem('userAddress')});
9         contratado._ultimoCheckeo = this.secondsToDateString(contratado.
  _ultimoCheckeo);
10        contratado._state = this.contratoStateToString(contratado._state);
11        this.listaContratados.push(contratado);
12        });
13    }
```

Listado 3.8. Muestra de información de contratos

```
1  <ion-col *ngFor="let prestamo of listaContratados;" size="3.1"  >
2       <ion-card >
3         <ion-card-content >
4           <ion-list >
5             <ion-item > <ion-label  class = "ion-text-center" >Estado: {{
  prestamo._state.toString() }}</ion-label> </ion-item>
6             <ion-item > <ion-label  class = "ion-text-center" >Deuda
  Restante: {{ prestamo._deudaRestante.toString() }}</ion-label> </ion-item>
7             <ion-item > <ion-label class = "ion-text-center" >Cuotas
  Restantes: {{ prestamo._cuotasRestantes.toString() }} </ion-label> </ion-
  item>
8             <ion-item > <ion-label class = "ion-text-center" >Cantidad
  Próxima Cuota: {{ prestamo._cuotaMensual.toString() }} </ion-label> </ion-
  item>
9             <ion-item > <ion-label class = "ion-text-center">Fin del plazo
  actual: {{ prestamo._ultimoCheckeo }}</ion-label> </ion-item>
10            <ion-item> <ion-label class = "ion-text-center">Penalización
  por Impago: {{ prestamo._penalizacionImpago.toString() }}%</ion-label> </
  ion-item>
11          </ion-list>
12          <ion-button (click)="pagarCuota(prestamo._direccion, prestamo.
  _cuotaMensual)">PAGAR CUOTA ({{prestamo._cuotaMensual.toString()}})</ion-
  button>
13        </ion-card-content>
14      </ion-card>
15 </ion-col>
```

Registro del estado antes de realizar transferencias

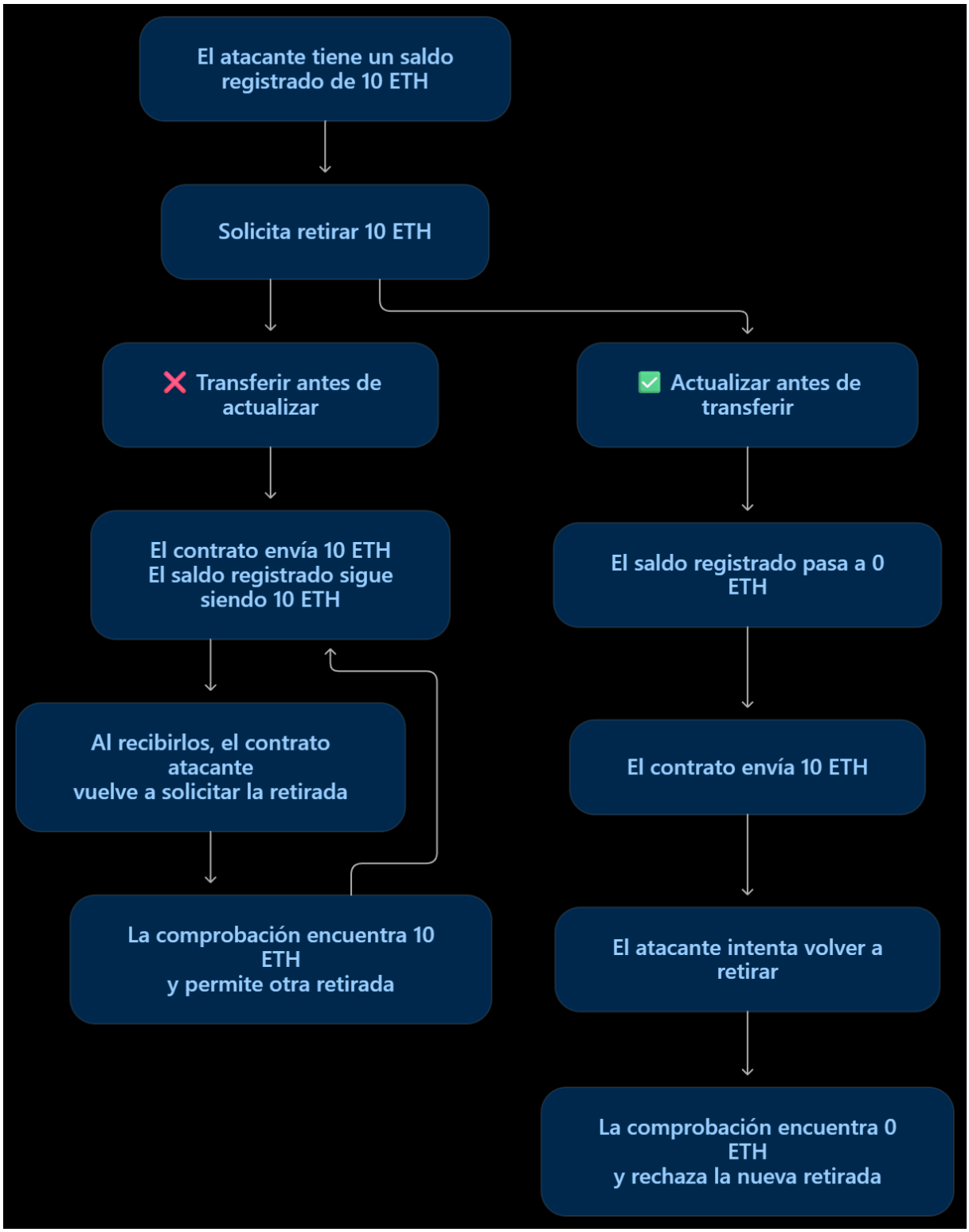
En Solidity es una buena práctica validar las condiciones de una operación y actualizar el estado del contrato antes de realizar transferencias de fondos.

Existe un tipo de ataque llamado *Ataque por reentrada*. En este ataque el riesgo se produce cuando un contrato inteligente transfiere fondos a otro contrato y este aprovecha la ejecución de su código para volver a invocar una función del contrato original antes de que termine la operación inicial. Si todavía no se han actualizado las variables de saldos, el atacante podría repetir una retirada utilizando un estado de las variables desactualizado y llegar a extraer fondos indebidamente.

Se puede ver cómo se realiza en este proyecto una transferencia de saldo en el fragmento de código 3.3.

Para un mayor entendimiento de este tipo de ataques se añade la figura 3.6 con un gráfico poniendo un ejemplo con una transferencia de 10 ETH.

Figura 3.6. Diagrama de un ataque de reentrada



3.5. Validación de resultados

3.5.1. Validación a través del uso de la aplicación del sistema

Tras el desarrollo del sistema de préstamos descentralizado, es necesario comprobar si cumple con los objetivos planteados y cubre los casos de uso definidos. Para ello, se ha realizado manualmente un flujo de uso de la interfaz de usuario y se presentan pruebas mediante capturas de pantalla de la interfaz y registros en la herramienta *Etherscan* para validar la ejecución de las transacciones y el correcto funcionamiento de los contratos inteligentes en la blockchain de Ethereum. Para comprender las siguientes imágenes es importante conocer los siguientes conceptos:

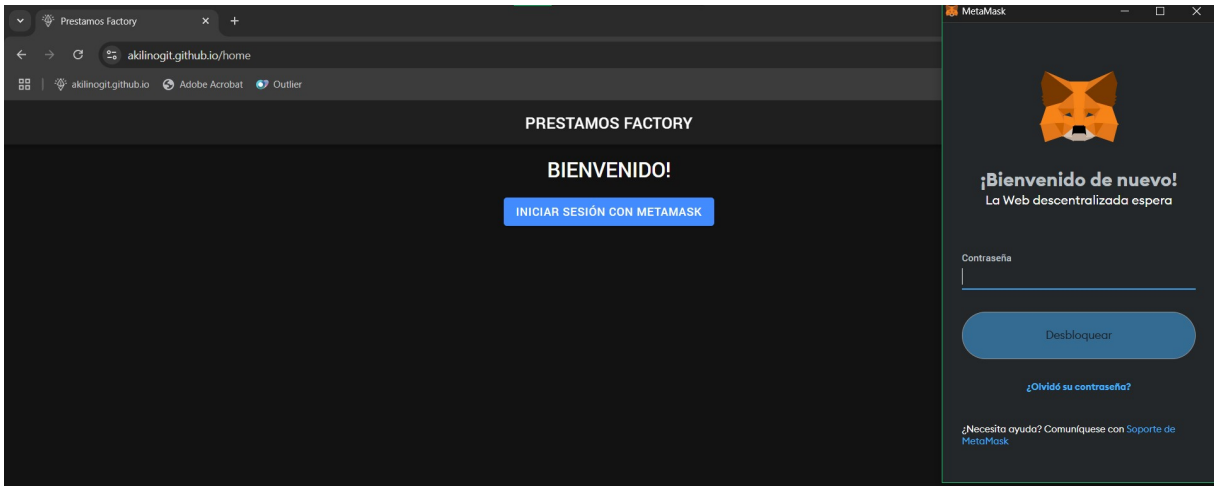
1. **Wei:** En la red Ethereum es la unidad más pequeña de valor para su criptomoneda ETH. Un ETH está compuesto por un total de 10^{18} wei.
2. Las cadenas de caracteres que comienzan por `'0x'` en la red Ethereum están reservadas como direcciones que pueden apuntar a usuarios, entidades o contratos inteligentes.

La interfaz reúne las funcionalidades del sistema en un único punto de acceso, en relación con el OP 2. Sin embargo, durante el desarrollo se ha priorizado la funcionalidad, por lo que la mejora de su accesibilidad y facilidad de uso se plantea como trabajo futuro.

Para la validación de resultados del OP 1 se pueden evaluar los distintos objetivos secundarios:

1. **OS 1:** La aplicación solicita a MetaMask la conexión de una cuenta y la autorización de las operaciones. La dirección permite asociar los préstamos a una cuenta; no acredita la identidad civil de su titular [33]. La figura 3.7 muestra este acceso a la aplicación.

Figura 3.7. Ejemplo de inicio de sesión con MetaMask



2. **OS 6:** El prototipo identifica a los usuarios mediante direcciones y no almacena datos personales de identificación. MetaMask permite autorizar conexiones y firmas sin entregar las claves privadas a la aplicación. Las direcciones y transacciones son públicas; esta elección no garantiza anonimato ni impide relacionar operaciones de una misma cuenta [33].

3. **OS 2:** Los contratos almacenan las condiciones y el estado de los préstamos, mientras la aplicación recupera y presenta la información. El prototipo evita incorporar datos personales al contrato.

4. **OS 3:** Las operaciones de consulta y presentación que no modifican el estado se realizan mediante llamadas de lectura y en la aplicación. Las funciones que interactúan con la red son sencillas y evitan procesamientos innecesarios. No obstante, la comprobación mensual de los pagos podría resultar costosa si aumenta el número de préstamos. Por ello, se propone estudiar mejoras que reduzcan el procesamiento necesario. No se presenta una comparación cuantitativa que demuestre una optimización global del gas.

5. Haciendo referencia al OS 4 se adjuntan las siguientes imágenes del panel de control de la entidad prestamista en el que se muestran 3 contratos de un usuario en curso antes y después de la comprobación mensual de los pagos de las cuotas. En ellas se pueden observar los distintos detalles:

   a) El balance de la entidad después del pago de cuota del usuario aumenta correctamente

b) Uno de los contratos no ha pagado correctamente la cuota y se le ha aplicado automáticamente la penalización por impago.

c) Uno de los contratos correctamente pagado ha reducido su deuda restante y el número de cuotas restantes.

d) En el caso del contrato que solo tenía una cuota restante, ya no se muestra en la dApp ya que no es necesario replicar más esta información.

Estas comprobaciones se pueden ver reflejadas con el cambio de información correspondiente en la figura que muestra la información previa a la comprobación mensual 3.8 y tras la comprobación mensual en la figura 3.9

Figura 3.8. Información de contratos previa a la comprobación mensual en el recuadro amarillo y balance de la entidad prestamista en el recuadro blanco.

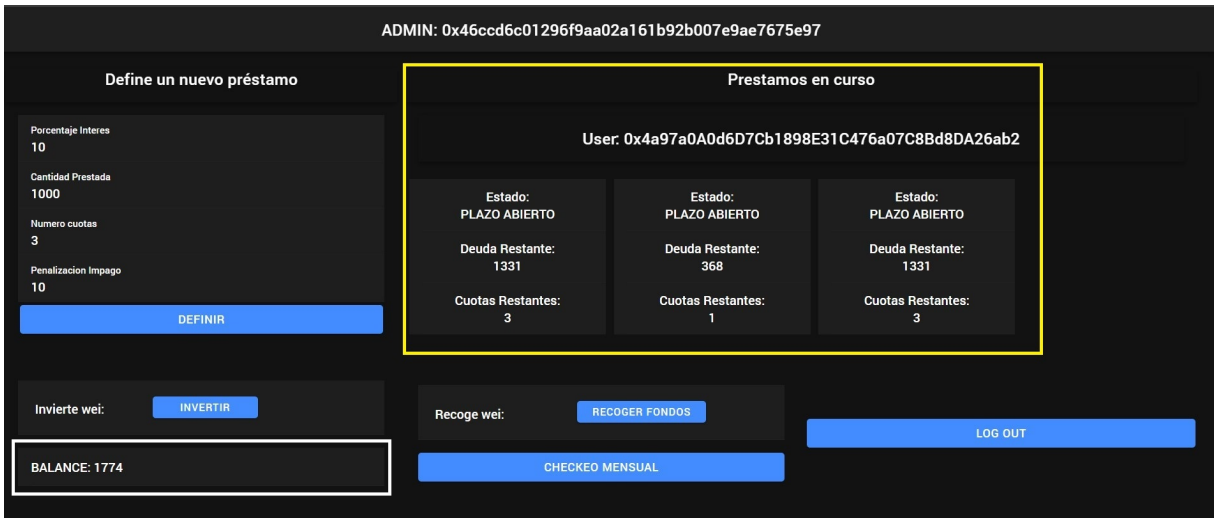
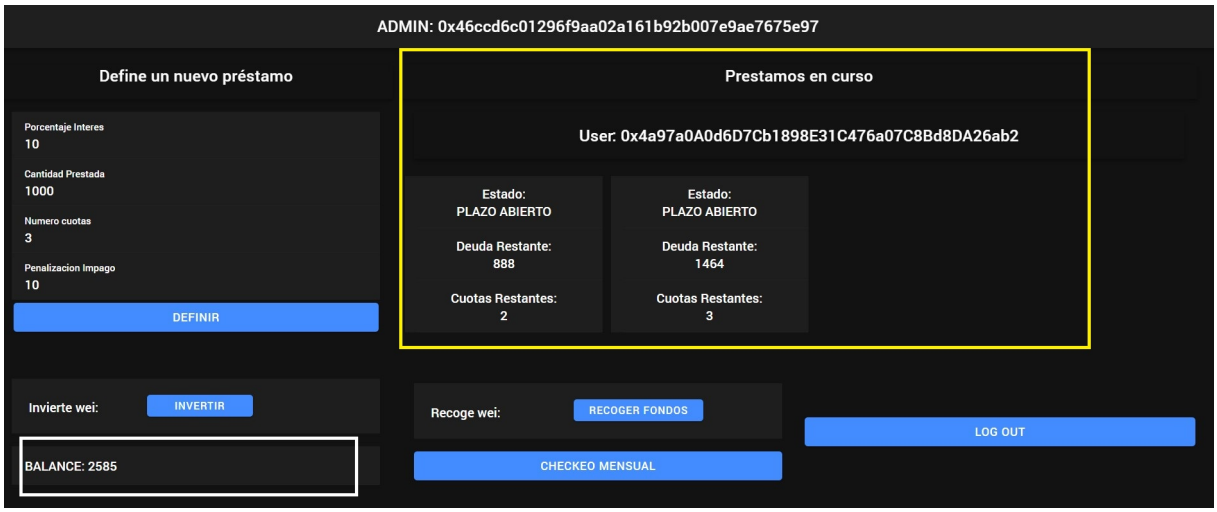


Figura 3.9. Información de contratos posterior a la comprobación mensual en el recuadro amarillo y balance de la entidad prestamista en el recuadro blanco.



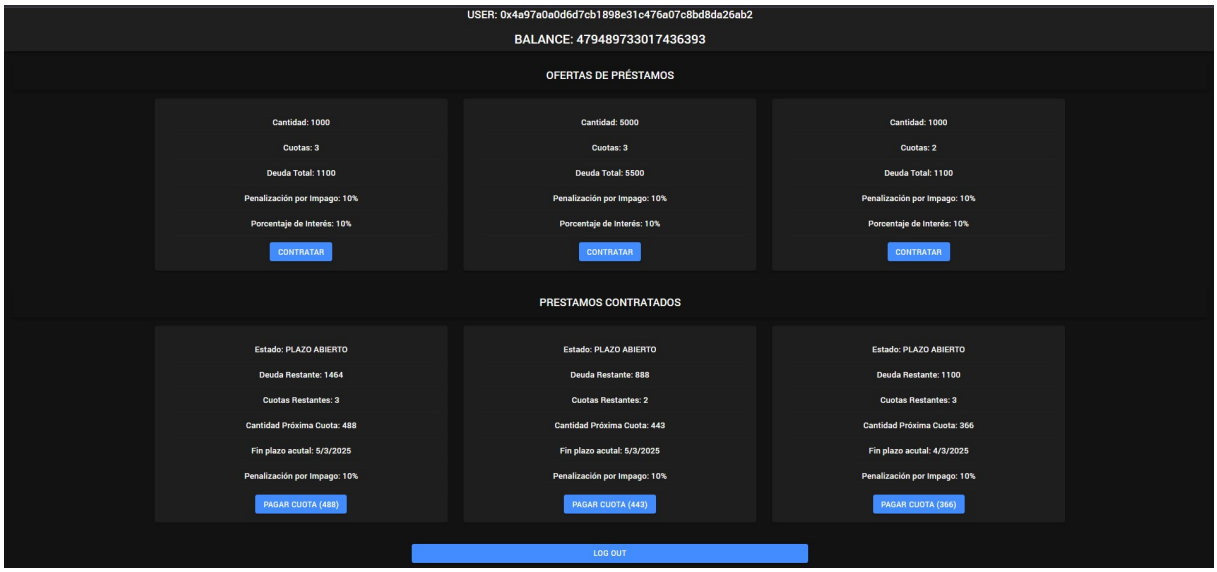
Las últimas características declaradas junto con la posibilidad de que la organización del sistema pueda recibir los wei del balance del sistema a través de la función *Recoger Fondos* nos llevan al alcance del OS 7, el cual busca encontrar la liquidez en este sistema.

6. En el OS 5 se contempla la posibilidad de personalizar los contratos accesibles para los distintos usuarios, esto se puede clasificar como inalcanzable para este proyecto debido a la necesidad de procesar grandes cantidades de datos de clientes para poder crear un algoritmo y un sistema que cubra esta necesidad.

   De momento existe la posibilidad de definir nuevos préstamos contratables en el panel de control de la entidad prestamista.

Para terminar de validar el OP 2 se adjunta captura de pantalla del panel de control del usuario en la figura 3.10 demostrando una interfaz unificada para el usuario.

Figura 3.10. Panel de control del usuario.



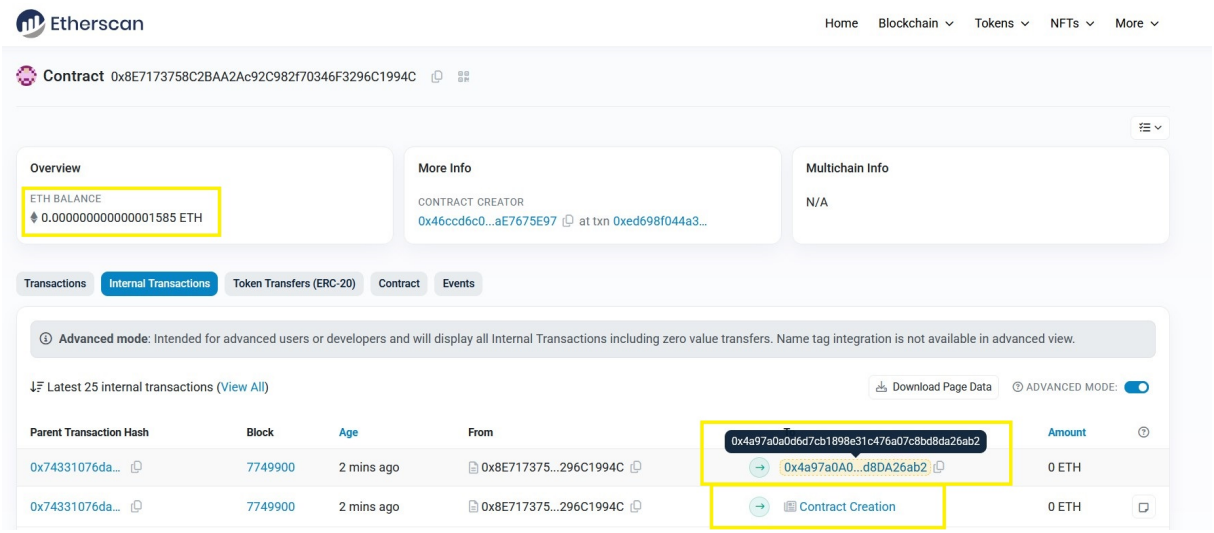
Como última prueba de validación manual de este proyecto se realiza un ejemplo de contratación de préstamo adjuntando distintas capturas de la web Etherscan Sepolia con las que se pueden comprobar distintos elementos relacionados con el proceso que corroboran la veracidad del préstamo a través de la red Sepolia.

Ejecución de préstamo de 1000 wei **Contrato principal:** En la anterior figura 3.9 se puede comprobar que el balance del contrato principal es de 2585 wei.

Balance del usuario: Después de la contratación del usuario, se puede ver a través de Etherscan Sepolia, tal y como se muestra en la figura 3.11, que:

- El balance del contrato ha disminuido 1000 wei
- Se ha registrado una transacción de creación de contrato (*Contract Creation*), correspondiente al nuevo préstamo.
- Aunque debido al redondeo de la página aparezca otra transacción de 0 ETH, viendo el balance se puede comprobar que es de 1000 wei y han sido transferidos a la dirección del usuario.

Figura 3.11. Información de Etherscan del registro en la red SepoliaETH del contrato inteligente principal

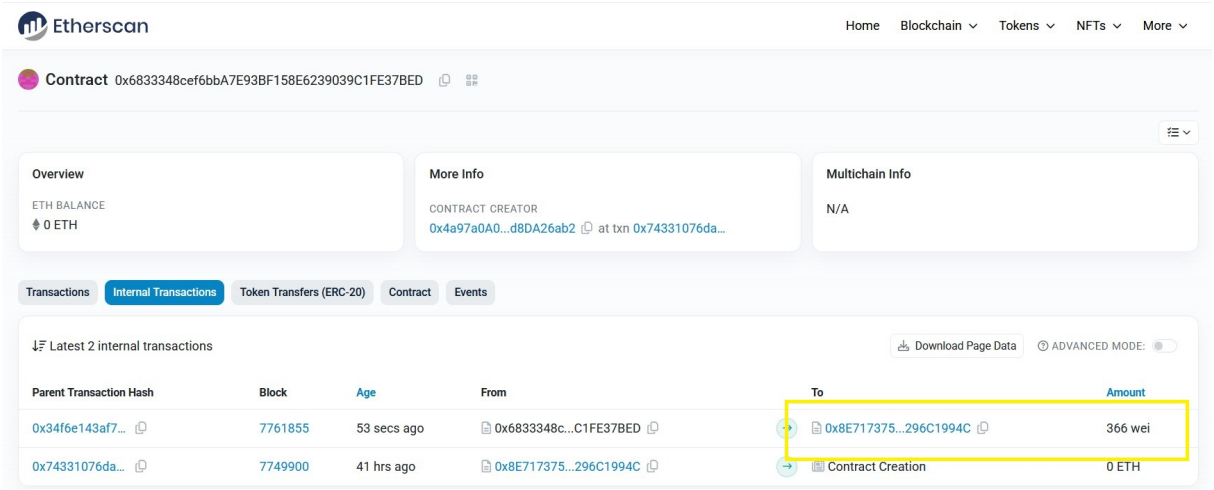


Se adjunta también otra captura de pantalla en la figura 3.12 de la información de la instancia generada del contrato inteligente *PrestamoCursando*

En la figura 3.12 también se puede observar dentro del recuadro amarillo la primera transferencia recibida por parte del usuario con la cuota mensual. Esta cantidad de 366 wei coincide correctamente con el atributo *Cantidad próxima cuota* del tercer préstamo contratado en el panel de usuario de la figura 3.10.

Con esta comprobación de función de pago en Etherscan Sepolia se da por validado que todas las funciones del proyecto se reflejan en la red otorgando veracidad e inmutabilidad al proyecto.

Figura 3.12. Registro en Sepolia en el que se puede ver una transacción o comprobante de pago Etherscan



3.5.2. Pruebas con Mocha

Ya se ha visto el correcto funcionamiento de la aplicación a través de la interfaz de usuario y la verificación de las transacciones en la red. Pero para garantizar el correcto funcionamiento de los contratos inteligentes en situaciones distintas al correcto uso de la aplicación, se han desarrollado distintas pruebas unitarias utilizando la herramienta entorno de pruebas Mocha.

Estas pruebas tienen como objetivo:

1. Comprobar el funcionamiento de los contratos inteligentes.
2. Verificar que las operaciones de préstamos funcionan correctamente.
3. Detectar errores en condiciones normales y situaciones no permitidas.
4. Comprobar las restricciones de acceso y los cambios de estado.

Red de pruebas local: Para evitar utilizar una red pública se han desplegado los contratos sobre una red local utilizando la herramienta Ganache [22]. Esto evita que aparezcan transacciones de prueba exponiendo información sensible del funcionamiento del sistema en la red pública y permite la repetición de escenarios que son difíciles de reproducir en un entorno real, como que un usuario no pague su cuota mensual y se le aplique la penalización correspondiente.

Cuentas utilizadas para las pruebas: La herramienta de creación de red local virtual es la encargada de generar distintas cuentas con las que interactuar en la red. Se han utilizado 3 cuentas para realizar las pruebas unitarias:

1. Una cuenta propietaria del sistema
2. Dos cuentas de usuario que solicitan el préstamo

Contratos inteligentes puestos a prueba: Las pruebas se han hecho sobre el motor de este sistema, es decir, sobre los contratos inteligentes que definen el funcionamiento del sistema de préstamos. Estos contratos se pueden ver en el anexo A y son los ya comentados *PrestamosFactory* y *PrestamoCursando*.

Tecnología de las pruebas: Las pruebas se han desarrollado utilizando JavaScript, el entorno de pruebas Mocha y el módulo `assert` de Node.js para las aserciones. Estas pruebas se pueden encontrar en el anexo B.

Desarrollo de las pruebas: Variables previas y condiciones iniciales: Previo al desarrollo puro de las pruebas, antes de cada ejecución se deben dar las condiciones iniciales para que las pruebas puedan ejecutarse correctamente. El proceso de preparación de las pruebas consiste en desplegar los contratos en la red local y establecer las condiciones iniciales para cada prueba. Para ello se han realizado los siguientes pasos:

1. **Compilación en local de los contratos inteligentes:** A través de la herramienta *Truffle* se compilan los contratos inteligentes previamente para que la red sea capaz de entender sus funciones. Esta compilación genera un archivo .json con las funciones del contrato inteligente parametrizadas para que la red sea capaz de interactuar con él.
2. **Obtención de cuentas:** Se obtienen las cuentas necesarias para realizar las pruebas.
3. **Despliegue del contrato en la red:** En el desarrollo de las pruebas se hace uso del archivo .json de la compilación y se despliega el contrato a testear en la red local de pruebas.

Listado 3.9. Despliegue de contrato en la red local

```
                    describe('Prestamos factory test', () => {

                    before( async() => {
                        accounts = await web3.eth.getAccounts();
                        factory = await new web3.eth.Contract(
prestamosFactoryBuild.abi)
                            .deploy({data: prestamosFactoryBuild.evm.
bytecode.object})
                            .send({ from: accounts[0], gas: 6721975});

                        console.log('contract address:'+ factory.
options.address);
                        console.log('Owner: ' + accounts[0]);


                    } );


```

4. **Establecimiento del estado inicial:** En el caso de las pruebas para los *PrestamoCursando*, se establecen primero las variables previas necesarias para la contratación de un préstamo. Estas son:

   a) **Introducción de fondos:** El sistema de préstamos necesita fondos previos para que se le puedan solicitar préstamos.

   b) **Creación de un tipo de préstamo:** Se crea un tipo de préstamo con las características necesarias para la prueba.

   c) **Creación de contratos de préstamo y obtención de sus direcciones:** De esta manera se obtiene la forma de acceder a los contratos sobre los que se ejecutarán las pruebas.

Listado 3.10. Parametrización de préstamos en curso

```
  describe('Prestamos Cursando test', async () => {
  //INICIALIZAR PRESTAMOS EN CURSO
  before(async() => {
      //Inicializar cuentas
      accounts = await web3.eth.getAccounts();

```

```
7
8         //Inicializar contract factory
9         factory = await new web3.eth.Contract(prestamosFactoryBuild.
    abi)
10            .deploy({data: prestamosFactoryBuild.evm.bytecode.object})
11            .send({ from: accounts[0], gas: 6721975});
12
13
14        //Invertir para poder crear nuevos contratos
15        await factory.methods.invertir().send({
16                from: accounts[0],
17                value: web3.utils.toHex(10000)
18            });
19        //Definir préstamo
20        await factory.methods.definirPrestamo(10,1000,2,10).send({from:
     accounts[0],
21            gas: web3.utils.toHex(2000000)});
22        //Contratar préstamos
23        await factory.methods.contratarPrestamo(accounts[1],0).send({
    from: accounts[1],
24            gas: web3.utils.toHex(2000000)});
25        await factory.methods.contratarPrestamo(accounts[2],0).send({
    from: accounts[2],
26            gas: web3.utils.toHex(2000000)});
27
28        [prestamo1address]=await factory.methods.verContratos(accounts
    [1]).call({from: accounts[1]});
29        [prestamo2address]=await factory.methods.verContratos(accounts
    [2]).call({from: accounts[2]});
30
31        prestamo1 = await new web3.eth.Contract(prestamoCursandoBuild.
    abi,prestamo1address);
32        prestamo2 = await new web3.eth.Contract(prestamoCursandoBuild.
    abi,prestamo2address);
33
34      });
35
```

5. **Descripción de las pruebas:** El contenido de las pruebas se puede ver en los dos ficheros del anexo B y se describen a continuación:

   a) **Pruebas del `PrestamosFactory`**

   En `test_prestamos_factory.js` se comprueba principalmente el **funcionamiento de PrestamosFactory y sus restricciones**:

1) **Despliegue del contrato**: se verifica que el `PrestamosFactory` se despliega correctamente y obtiene una dirección.
2) **Control de acceso**: se comprueba que determinadas funciones únicamente pueden ser ejecutadas por el `owner`.
3) **Ingreso de fondos**: se comprueba que `invertir()` incrementa correctamente el saldo del Factory.
4) **Consulta de saldo**: se verifica que `getBalance()` devuelve el saldo real del contrato.
5) **Estado inicial**: se comprueba que inicialmente no existen clientes, tipos de préstamo ni préstamos en curso.
6) **Retirada de fondos**: se verifica que `recolectarFondos()` reduce correctamente el saldo del Factory.
7) **Validación de parámetros**: se prueban valores incorrectos para los porcentajes de intereses y penalizaciones, comprobando que las operaciones son rechazadas.
8) **Creación de tipos de préstamo**: se comprueba que `definirPrestamo()` crea correctamente un nuevo producto y genera el evento correspondiente.
9) **Consulta de tipos de préstamo**: se verifica que los datos almacenados coinciden con los valores introducidos.
10) **Contratación de préstamos**: se comprueba tanto la contratación correcta como situaciones en las que la operación debe fallar.
11) **Consulta de préstamos de un usuario**: se verifica que `verContratos()` devuelve los contratos correspondientes.
12) **Intento de eliminar un préstamo no finalizado**: se comprueba que esta operación no está permitida.

b) **Pruebas del `PrestamoCursando`**

El segundo archivo prueba el **ciclo de vida de un préstamo individual**.

Se parte de una situación preparada con:

1) Factory desplegado.
2) Fondos introducidos.
3) Tipo de préstamo creado.
4) Dos préstamos contratados por usuarios diferentes.

A partir de ahí se prueban:

1) **Control de acceso a la información**: un usuario diferente al prestatario no puede consultar la información privada del préstamo.
2) **Consulta de información del préstamo**: se comprueban importe prestado, deuda, cuota mensual, número de cuotas, estado, prestatario, etc.
3) **Restricción temporal**: se comprueba que no se puede realizar el chequeo mensual antes de que haya transcurrido el tiempo establecido.
4) **Pago de una cuota**: se realiza un pago y se comprueba que:
   a) disminuye el número de cuotas restantes;
   b) disminuye la deuda;
   c) cambia el estado del préstamo.
5) **Transferencia de fondos**: se verifica que el dinero pagado vuelve correctamente al `PrestamosFactory` y que su balance es el esperado.
6) **Chequeo mensual**: se simula el paso del tiempo y se comprueba que el chequeo:
   a) actualiza el estado del préstamo;
   b) aplica una penalización al préstamo que no ha sido pagado.
7) **Finalización del préstamo**: se paga la última cuota y se comprueba que el préstamo pasa al estado de finalizado.
8) **Eliminación de préstamos finalizados**: después del chequeo mensual se comprueba que el contrato finalizado deja de aparecer entre los préstamos activos del usuario y que se actualiza la lista de clientes.

6. **Comprobación de las pruebas:** Las aserciones del módulo `assert` de Node.js comparan los resultados obtenidos con los esperados. Mocha permite organizar las pruebas y presentar el resultado de su ejecución.

Resultados: Los archivos del anexo B describen las comprobaciones de acceso, pagos, penalizaciones y finalización de préstamos. Las capturas de la validación manual aportan ejemplos concretos de operaciones. Esta copia del proyecto no incluye una salida de Mocha asociada a una versión identificada de los contratos ni los artefactos compilados necesarios para reproducirla. Por tanto, estos materiales documentan los escenarios previstos, pero no acreditan que todas las pruebas automáticas de la versión incluida hayan superado una ejecución completa.

Se destacan dos aspectos de la implementación documentada:

1. La función `setIndex` actualiza el índice de un préstamo. En el código del anexo está restringida mediante `onlyContractFactory`, que comprueba que el emisor sea el contrato principal. El acceso arbitrario descrito en versiones anteriores del texto no corresponde al código incluido; tampoco se aporta un registro que permita fechar una eventual corrección.

2. El chequeo mensual exige esperar el plazo configurado. En el código del anexo, `PLAZO` vale diez segundos para las pruebas, frente al intervalo mensual previsto para una explotación real. Las esperas de las pruebas deben tenerse en cuenta al configurar el tiempo de espera de Mocha.

3.5.3. Validación del proyecto después de pruebas

Las comprobaciones manuales documentadas permiten valorar la viabilidad técnica del prototipo. Su utilización en un entorno real requeriría desarrollar los mecanismos adicionales descritos en el apartado de trabajo futuro y completar una validación reproducible de la versión que se desplegara.

3.5.4. Viabilidad económica

Tras estudiar los gastos de gas en los registros de Etherscan Sepolia se puede comprobar que los gastos de contratar un préstamo dependen mucho del precio del gas en el momento de contratación, el cual varía de forma muy dinámica dependiendo de la congestión de la red. En el proceso de validación se han ejecutado transferencias de préstamo con costes desde 0,46 dólares hasta 30,49 dólares. Esto puede crear un cuello de botella de entrada de préstamos en caso de que se dependa en exceso de esta variable obligando a la gestión del sistema a retrasar contratos y operaciones cuando el coste sea muy alto o directamente pasar los costes a los clientes. El procesamiento del chequeo mensual aumenta con el número de contratos. Si este incremento coincide con un coste elevado del gas, la operación puede resultar demasiado costosa. Por ello, sería necesario optimizar la forma en que se realiza la comprobación.

4. Conclusiones

En este Proyecto Fin de Grado se ha desarrollado un sistema de préstamos descentralizado basado en la red Ethereum. Tras el desarrollo se puede llegar a las siguientes conclusiones:

- Se puede afirmar que se ha obtenido una aplicación capaz de conectarse con una red blockchain y de realizar transacciones en ella.

  Gracias al registro de estas transacciones y cierta lógica aplicada a esta información, se puede componer un sistema de creación y control de préstamos digitales en el que la contratación de nuevos préstamos queda totalmente automatizada.

- Es posible reunir las herramientas de gestión en una única interfaz de usuario, capaz de conectarse con Ethereum y solicitar las operaciones necesarias para administrar los préstamos.

- Aunque para un sistema aplicado en la vida real hiciera falta la implementación de distintos desarrollos, los contratos inteligentes desplegados en la red definen correctamente la lógica aplicada a la información necesaria para la concesión de préstamos.

- El correcto registro de esta información en la red blockchain proporciona al sistema las características de seguridad, inmutabilidad y veracidad necesarias para un sistema de esta índole.

- Existe un plan de validación que define escenarios para comprobar las capacidades descritas para este sistema.

- Tras un estudio del impacto social se puede concluir que la adopción de tecnologías DeFi y contrato inteligente tiene el potencial de transformar el acceso a servicios financieros y dependiendo del uso que se dé a esta tecnología se puede aumentar o disminuir la inclusión financiera.

4.1. Trabajo futuro

En el desarrollo de este proyecto se ha podido comprobar que un sistema de préstamos descentralizado es posible, pero, para que un proyecto como este llegue a una situación de producción real es necesario realizar una serie de desarrollos para cumplir con necesidades básicas en un sistema de préstamo real.

La mejora prioritaria es incorporar garantías de devolución en caso de impago, como se recoge en el caso de uso futuro CU12. Aunque el sistema aplica penalizaciones, estas no garantizan por sí mismas la recuperación del capital prestado. Se plantean dos posibles desarrollos:

- Creación de sistema de colaterales: en un futuro será de obligado cumplimiento que el usuario que solicite un préstamo añada al sistema alguna garantía, la cual puede estar en forma de NFT, y en caso de que el usuario se declare en situación de impago, este NFT pasaría automáticamente a ser propiedad del sistema
- Creación de sistema de avales: En situaciones en las que el usuario no pueda añadir esa garantía o colateral se podrá añadir al contrato un tercero capaz de añadir un colateral al contrato e incluso encargarse de las cuotas en caso de que el usuario principal no lo haga

El nuevo sistema de garantías también deberá incluir un sistema de validación de solvencia, tal y como se habla en el Caso de uso futuro CU14, lo que aportará mayor solidez a la garantía de devolución.

Durante los pruebas de funcionamiento del sistema se pudieron comprobar las ineficiencias de código en la red. Sobre todo, se debe evitar recorrer todos los contratos existentes una vez al mes para comprobar si han sido pagados o no.

Para ello el sistema se puede apoyar en elementos externos a la red, como una base de datos temporal en la que puede quedar reflejado cada pago de la mensualidad y mensualmente ejecutar las penalizaciones en la red con los contratos que no aparecen en esta base de datos, aunque sea una base de datos externa, su uso no rompe con ninguna de las características en las que se apoya el sistema ya que solo sería un apoyo técnico, manteniendo siempre reflejada en la red la situación financiera del sistema.

Como se explica en la sección 3.2.1, MiCA fue adoptado en 2023 y su aplicación general

comenzó el 30 de diciembre de 2024. Una explotación real requiere determinar el encaje normativo del servicio y las obligaciones aplicables a la entidad prestamista [31].

En el objetivo principal 2 se establece que este proyecto debe contener una interfaz de usuario unificada, fácil e intuitiva. Aunque sí se ha desarrollado una interfaz unificada sería necesario crear una interfaz más intuitiva para el usuario.

Con menos prioridad se deberán desarrollar sistemas que utilicen Oráculos tal y como se habla en el Caso de uso Futuro CU13, este sistema junto a la comprobación de solvencia económica pueden servir para una correcta definición de los tipos de préstamo y condiciones disponibles en el sistema.

4.2. Impactos sociales y ambientales del desarrollo de este sistema a gran escala

La adopción masiva de un sistema de préstamos descentralizado basado en blockchain y contratos inteligentes podría generar importantes transformaciones en el ámbito financiero y social. La eliminación de intermediarios permitiría un acceso más democrático a los servicios financieros, especialmente en regiones con sistemas bancarios limitados o restrictivos.

Sin embargo, este modelo también presenta retos significativos en términos de inclusión digital, regulación y sostenibilidad ambiental.

4.2.1. Impactos sociales

Sin embargo, la reducción de intermediarios no garantiza por sí sola una mayor inclusión financiera. Su impacto social dependería de las condiciones de acceso al crédito, de las competencias digitales de los usuarios y de los mecanismos de protección disponibles [34].

1. La posibilidad de otorgar préstamos sin la necesidad de bancos tradicionales podría beneficiar a millones de personas no bancarizadas en economías emergentes. Sin embargo, la Organización para la Cooperación y el Desarrollo Económico

(OCDE) señala que el acceso a estos servicios requiere conocimientos sobre criptomonedas, carteras digitales y plataformas DeFi, lo que podría ampliar la brecha digital. Además, la responsabilidad de los usuarios sobre la gestión de sus propios activos puede dificultar la utilización de estas herramientas [34].

Por tanto, el acceso técnico a una plataforma no equivale a una inclusión financiera efectiva. En el sistema desarrollado, sería necesario facilitar la comprensión de su funcionamiento y del uso de una cartera para reducir estas barreras [34].

2. La descentralización del sistema podría entrar en conflicto con regulaciones financieras existentes. Gobiernos y bancos centrales podrían imponer restricciones al uso de plataformas DeFi, especialmente en países donde el control financiero es una herramienta de política económica. Además, la anonimidad en las transacciones podría plantear problemas relacionados con el financiamiento ilícito si no se establecen mecanismos de verificación de identidad adecuados.

3. La automatización de los préstamos a través de contratos inteligentes podría reducir la demanda de intermediarios financieros, afectando a trabajadores del sector bancario y generando resistencia por parte de instituciones tradicionales. Sin embargo, también abriría oportunidades en nuevas áreas tecnológicas, creando demanda de profesionales en blockchain, ciberseguridad y programación de contratos inteligentes.

4. **Acceso efectivo al crédito.** El BIS advierte que muchos préstamos DeFi requieren aportar garantías cuyo valor supera el importe solicitado. Esta exigencia limita su utilidad para personas que carecen de patrimonio previo y necesitan financiación [35].

4.2.2. Impactos ambientales

1. Ethereum, aunque ha reducido su impacto ambiental tras su transición de Proof of Work (PoW) a Proof of Stake (PoS), sigue dependiendo de una infraestructura global de nodos y validadores que consumen energía. A gran escala, este sistema de préstamos podría contribuir al aumento de la demanda energética de los centros de datos y redes descentralizadas, lo que requeriría soluciones de eficiencia energética y compensación de emisiones.

Si bien un sistema de préstamos descentralizado tiene el potencial de democratizar el acceso financiero y reducir costes, también presenta desafíos sociales y ambientales que

deben abordarse antes de una implementación a gran escala. Para mitigar estos impactos, sería fundamental desarrollar estrategias de educación digital y establecer mecanismos de regulación equilibrados para mejorar la eficiencia energética del sistema.

Solo con un enfoque responsable y sostenible se podrá garantizar que este tipo de innovación beneficie a la sociedad sin comprometer el bienestar ambiental.

4.2.3. Objetivos de desarrollo sostenible. Agenda 2030

La Agenda 2030 para el Desarrollo Sostenible es un plan global adoptado en 2015 por los Estados Miembros de la ONU con el objetivo de erradicar la pobreza, proteger el planeta y garantizar la prosperidad para todas las personas. Para lograrlo, se establecieron 17 Objetivos de Desarrollo Sostenible (ODS), que abarcan áreas clave como la educación, la igualdad de género, la acción climática, la innovación y el crecimiento económico, entre otras.

Estos objetivos buscan equilibrar el desarrollo social, económico y ambiental, promoviendo un modelo sostenible que asegure el bienestar de las generaciones actuales y futuras. Además, llaman a la colaboración entre gobiernos, empresas y la sociedad civil para alcanzar metas concretas antes del año 2030.

En la figura 4.1 se pueden ver estos objetivos:

Figura 4.1. Objetivos de desarrollo sostenible de la Agenda 2030 de las Naciones Unidas



Se puede concluir que este proyecto tiene un gran potencial de influir positivamente en los objetivos relacionados con el ámbito financiero como los ODS 8 y 9 ya que supone la creación de una nueva infraestructura en el desarrollo de actividades económicas favoreciendo nuevas oportunidades de crecimiento económico.

El desarrollo de plataformas DeFi podría contribuir a reducir las desigualdades y favorecer la justicia social (ODS 10). Sin embargo, su impacto depende del uso de la tecnología. Si no se combate la brecha digital ni se evitan nuevas barreras de acceso, estas plataformas podrían aumentar las desigualdades en lugar de reducirlas.

En relación con el ODS 16, la transparencia del registro de transacciones puede facilitar la detección de irregularidades. Su utilidad social depende también de la existencia de mecanismos de supervisión y de una aplicación efectiva de la justicia.

En el apartado ambiental (ODS 11, 12 y 13), aunque es cierto que tras el cambio de validación de bloques en la red a través de puro procesamiento y consumo eléctrico (PoW) por un mecanismo de consenso basado en la participación de capital en la red (PoS) se redujo de manera drástica el consumo de electricidad, deben considerarse otros efectos ambientales, como la gestión de los residuos de la infraestructura. Parte de las mejoras de productividad y de los recursos obtenidos podría destinarse a mitigar estos

impactos.

A. Código fuente de los contratos inteligentes

En este anexo se presenta el código fuente de los contratos inteligentes desarrollados para el sistema de préstamos descentralizado basado en la red Ethereum.

A.1. PRESTAMOS_FACTORY.sol

El archivo `PRESTAMOS_FACTORY.sol` incluye el contrato principal `PrestamosFactory`, encargado de gestionar los tipos de préstamo y la creación de nuevos contratos, y el contrato `PrestamoCursando`, que representa un préstamo individual en curso.

Listado A.1. Código fuente de PRESTAMOS_FACTORY.sol

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.8.0 <0.9.0;
import "@openzeppelin/contracts/utils/math/SafeMath.sol";
//SafeMath.sol no aparece con npm install, importado manualmente
contract PrestamosFactory {

    using SafeMath for uint256;

    address payable private owner;
    //Posibles préstamos a contratar
    TipoContrato[] tiposContrato;

    mapping(address => PrestamoCursando[])  prestamosCursando;
    address[] clientes;

    struct TipoContrato {
        uint256 index;
        uint256 porcentajeInteres;
        uint256 cantidad;
        uint256 deudaTotal;
```

```
21          uint256 cuotas;
22          uint256 penalizacionImpago;
23      }
24
25      event PrestamoDefinido(TipoContrato tipoContrato);
26      event NuevoPrestamoCursando(address indexed direccionContrato, address
    indexed prestatario);
27      event FondosRecibidos(address direccion,uint256 cantidad);
28
29      modifier onlyOwner() {
30          require(owner == msg.sender,"Only Owner");
31           _;
32      }
33
34      constructor(){
35          owner = payable(msg.sender);
36      }
37
38      receive() external payable {
39          invertir();
40      }
41
42      function invertir() public payable {
43          emit FondosRecibidos(msg.sender, msg.value);
44      }
45
46      function getBalance() public view onlyOwner returns(uint256) {
47
48          return address(this).balance;
49      }
50      function getClientes() public view onlyOwner  returns(address[] memory)
    {
51          return clientes;
52
53      }
54
55      function recolectarFondos(uint256 amount) public  onlyOwner {
56          require(address(this).balance >= amount);
57          owner.transfer(amount);
58
59      }
60
61      function verTiposContrato() public view returns (TipoContrato[] memory)
    {
62          return tiposContrato;
```

```
63    }
64
65
66    function verContratos(address _prestatario) public view returns (
      PrestamoCursando[] memory){
67        require((msg.sender == owner)||(msg.sender == _prestatario), "Top
      Secret");
68        return (prestamosCursando[_prestatario]);
69    }
70
71
72    function definirPrestamo(uint8 _porcentajeInteres, uint256 _cantidad,
      uint8 _cuotas, uint8 _penalizacionImpago) public onlyOwner {
73
74        require(_penalizacionImpago >= 0 && _penalizacionImpago <= 100, "La
       penalizacion tiene que ser un porcentaje");
75        require(_porcentajeInteres >= 0 && _porcentajeInteres <= 100, "El
      interes tiene que ser un porcentaje");
76
77
78        uint256 _deudaTotal = _cantidad + _cantidad.mul(_porcentajeInteres).
      div(100);
79
80        TipoContrato memory nuevoPrestamo = TipoContrato({
81            index: 0 ,
82            porcentajeInteres: _porcentajeInteres,
83            cantidad: _cantidad,
84            deudaTotal: _deudaTotal,
85            cuotas: _cuotas,
86            penalizacionImpago: _penalizacionImpago
87        });
88
89        tiposContrato.push(nuevoPrestamo);
90        //Guardo el index dentro del Objeto préstamo para mayor facilidad
      de acceso
91        tiposContrato[tiposContrato.length - 1].index = tiposContrato.
      length-1;
92        emit PrestamoDefinido(nuevoPrestamo);
93
94    }
95
96
97
98
99    function contratarPrestamo(address payable _prestatario, uint8
```

```
indexTipoContrato) public {
100
101         require(msg.sender == _prestatario, "No se puede contratar en
    nombre de otros");
102
103         TipoContrato memory tipoPrestamo = tiposContrato[indexTipoContrato];
104         require(address(this).balance >= tipoPrestamo.cantidad,"Prestamo no
     disponible");
105
106         //Compruebo si la cuenta tiene otro contrato contratado
107         if(prestamosCursando[_prestatario].length == 0){
108             clientes.push(_prestatario);
109         }
110         //Guardo index para facilitar acceso
111         uint256 _indexNuevoPrestamo = prestamosCursando[_prestatario].
    length;
112         // Genero un nuevo contrato Prestamo asociado a la dirección msg.
    sender
113         PrestamoCursando newPrestamo = new PrestamoCursando(
114             _indexNuevoPrestamo,
115             _prestatario,
116             owner,
117             tipoPrestamo.cuotas,
118             tipoPrestamo.cantidad,
119             tipoPrestamo.deudaTotal,
120             tipoPrestamo.penalizacionImpago
121         );
122
123         //Enviar dinero a usuario
124         emit NuevoPrestamoCursando(address(newPrestamo), _prestatario);
125         prestamosCursando[_prestatario].push(newPrestamo);
126         _prestatario.transfer(tipoPrestamo.cantidad);
127
128     }
129
130     function  eliminarPrestamoFinalizado(address user, uint index) external
     {
131         PrestamoCursando eliminar = prestamosCursando[user][index];
132         PrestamoCursando.State state = PrestamoCursando(eliminar).
    mostrarEstado();
133         require( state == PrestamoCursando.State.FINALIZADO , "Contrato no
    finalizado");
134         //Se copia el último elemento del array y se le cambia el índice
135         prestamosCursando[user][index] = prestamosCursando[user][
```

```
  prestamosCursando[user].length - 1];
136         prestamosCursando[user][index].setIndex(index);
137         //Se elimina el elemento copiado
138         prestamosCursando[user].pop();
139 
140         //En caso de que el cliente deje de tener contratos con nosotros le
     eliminamos de la lista de clientes
141         if( prestamosCursando[user].length == 0){
142             for(uint i = 0 ; i < clientes.length; i ++ ) {
143                 if (clientes[i] == user) {
144                     clientes[i] = clientes[clientes.length - 1];
145                     break;
146                 }
147             }
148             clientes.pop();
149         }
150 
151     }
152 
153     }
154 
155 contract PrestamoCursando {
156 
157     using SafeMath for uint256;
158 
159     uint index;
160     enum State { PAGADO, ABIERTO, FINALIZADO }
161     State state;
162     address prestatario;
163     address entidadFinanciera;
164     address payable prestamosFactory;
165     uint256 cuotasRestantes;
166     uint256 cantidadPrestada;
167     uint256 deudaRestante;
168     uint256 cuotaMensual;
169     uint256 penalizacion;
170     uint256 ultimoCheckeo;
171 
172     struct InfoContrato {
173         address _direccion;
174         uint256 _index;
175         address _prestatario;
176         State _state;
177         uint256 _cantidadPrestada;
178         uint256 _deudaRestante;
```

```
179         uint256 _cuotaMensual;
180         uint256 _cuotasRestantes;
181         uint256 _penalizacionImpago;
182         uint256 _ultimoCheckeo;
183     }
184 
185     //2629800 = 1 MES;
186     uint256 public constant PLAZO = 10;
187 
188     modifier onlyFinancialEntity() {
189         require(entidadFinanciera == msg.sender,"Only financial entity");
190          _;
191     }
192 
193     modifier onlyContractFactory() {
194         require(prestamosFactory == msg.sender,"Only contract Factory");
195          _;
196     }
197 
198     modifier hasBeenOneMonth() {
199         require(block.timestamp - ultimoCheckeo >= PLAZO, "Deadline has not
 yet expired" );
200         _;
201     }
202 
203     constructor(uint256 _index, address _prestatario, address _entidad,
uint256 _cuotasRestantes, uint256 _cantidad, uint256 _deudaTotal, uint256
_penalizacion){
204         index = _index;
205         state = State.ABIERTO;
206         prestatario = _prestatario;
207         entidadFinanciera = _entidad;
208         prestamosFactory = payable(msg.sender);
209         cuotasRestantes = _cuotasRestantes;
210         cantidadPrestada = _cantidad;
211         deudaRestante = _deudaTotal;
212         penalizacion = _penalizacion;
213         cuotaMensual = calculoCuotaMensual();
214         ultimoCheckeo = block.timestamp;
215     }
216 
217     function mostrarInfo() external view returns (InfoContrato memory){
218         require((msg.sender == entidadFinanciera || msg.sender ==
prestatario), "Top Secret");
219 
```

```
220         InfoContrato memory infoContrato = InfoContrato({
221             _direccion: address(this),
222             _index: index,
223             _prestatario: prestatario,
224             _state: state,
225             _cantidadPrestada: cantidadPrestada,
226             _deudaRestante: deudaRestante,
227             _cuotaMensual: cuotaMensual,
228             _cuotasRestantes: cuotasRestantes,
229             _penalizacionImpago: penalizacion,
230             _ultimoCheckeo: ultimoCheckeo
231         });
232 
233         return infoContrato;
234     }
235 
236     function mostrarEstado() external view returns (State) {
237         return state;
238     }
239 
240 
241     function setIndex(uint _index) external onlyContractFactory {
242         index = _index;
243     }
244 
245     function calculoCuotaMensual() private view returns (uint256){
246         //La división entera deudaRestante/cuotasRestantes puede dejar un
    resto
247         //Esto implica que quede deuda después de pagar todas las cuotas
    por lo que este resto se añade a la cuota final
248         if(cuotasRestantes == 1) {
249             return  deudaRestante;
250             }
251         else {
252             return deudaRestante.div(cuotasRestantes);
253         }
254 
255     }
256 
257     function checkeoMensual() onlyFinancialEntity hasBeenOneMonth public {
258 
259         if(state == State.ABIERTO){
260             penalizarPrestamo();
261         }
262         else if(state == State.PAGADO){
```

```
263             state = State.ABIERTO;
264         }
265         else if (state == State.FINALIZADO){
266             eliminarContratoFinalizado();
267         }
268 
269         ultimoCheckeo = block.timestamp;
270 
271     }
272 
273     function eliminarContratoFinalizado() private {
274         PrestamosFactory(prestamosFactory).eliminarPrestamoFinalizado(
    prestatario, index);
275     }
276 
277     //Número de cuotas se mantiene
278     function penalizarPrestamo() private {
279         deudaRestante += deudaRestante.mul(penalizacion).div(100);
280         cuotaMensual = calculoCuotaMensual();
281     }
282 
283     receive() external payable {
284         pagarMensualidad();
285     }
286 
287     function pagarMensualidad() payable public {
288         require(msg.sender == prestatario, "Only borrower");
289         require(msg.value == cuotaMensual, "Send exact debt value");
290         require(state == State.ABIERTO, "Payment period not open");
291 
292         state = State.PAGADO;
293         deudaRestante -= cuotaMensual;
294         cuotasRestantes--;
295 
296 
297         if(cuotasRestantes == 1){
298             cuotaMensual = calculoCuotaMensual();
299          }
300 
301         if(cuotasRestantes == 0){
302          state = State.FINALIZADO;
303         }
304         transferirFondosAFactory(msg.value);
305 
306     }
```

```
307
308     //Por seguridad se pone a public para evitar fondos bloqueados en este
        contrato.
309     function transferirFondosAFactory(uint256 _amount) public {
310         prestamosFactory.transfer(_amount);
311     }
312
313
314
315 }
```

B. Código fuente de las pruebas

En este anexo se incluye el código fuente de las pruebas desarrolladas sobre los contratos inteligentes.

B.1. test_prestamos_cursando.js

Este archivo contiene las pruebas destinadas a verificar el comportamiento relacionado con los préstamos en curso.

Listado B.1. Pruebas de préstamos en curso

```
const assert = require('assert');
const ganache = require('ganache-cli');
const Web3 = require('web3');
const web3 = new Web3(ganache.provider());
const prestamosFactoryBuild = require ('../build/contracts/PrestamosFactory.
  json');
const prestamoCursandoBuild = require ('../build/contracts/PrestamoCursando.
  json');


describe('Prestamos Cursando test', async () => {
    //INICIALIZAR PRESTAMOS EN CURSO
    before(async() => {
        //Inicializar cuentas
        accounts = await web3.eth.getAccounts();


        //Inicializar contract factory
        factory = await new web3.eth.Contract(prestamosFactoryBuild.abi)
            .deploy({data: prestamosFactoryBuild.evm.bytecode.object})
            .send({ from: accounts[0], gas: 6721975});


```

```
22          //Invertir para poder crear nuevos contratos
23          await factory.methods.invertir().send({
24                  from: accounts[0],
25                  value: web3.utils.toHex(10000)
26              });
27          //Definir préstamo
28          await factory.methods.definirPrestamo(10,1000,2,10).send({from:
    accounts[0],
29              gas: web3.utils.toHex(2000000)});
30          //Contratar préstamos
31          await factory.methods.contratarPrestamo(accounts[1],0).send({from:
    accounts[1],
32              gas: web3.utils.toHex(2000000)});
33          await factory.methods.contratarPrestamo(accounts[2],0).send({from:
    accounts[2],
34              gas: web3.utils.toHex(2000000)});
35
36          [prestamo1address]=await factory.methods.verContratos(accounts[1]).
    call({from: accounts[1]});
37          [prestamo2address]=await factory.methods.verContratos(accounts[2]).
    call({from: accounts[2]});
38
39          prestamo1 = await new web3.eth.Contract(prestamoCursandoBuild.abi,
    prestamo1address);
40          prestamo2 = await new web3.eth.Contract(prestamoCursandoBuild.abi,
    prestamo2address);
41
42
43          console.log('Factory address:'+ factory.options.address);
44          console.log('Owner: ' + accounts[0]);
45          console.log('Prestamo1: '+ prestamo1.options.address);
46
47          console.log('--------');
48          console.log('||TEST||');
49          console.log('--------');
50
51      });
52
53      it('mostrar info desde cuenta externa', async() => {
54          try{
55              await prestamo1.methods.mostrarInfo().call({from: accounts[2]});

56              assert.fail(new TypeError('Sin acceso'));
57          }catch (error){
58
```

```
59          }
60      });
61
62      it('Mostrar info', async() => {
63          var expected = {
64              _direccion: '0xe08d37dF20bEad7007f406C847c99E367AFF4ea1',
65              _index: '0',
66              _prestatario: '0x9a9d0e305D5F97b5881055E12a9240AA1C0E8E4e',
67              _state: '1',
68              _cantidadPrestada: '1000',
69              _deudaRestante: '1100',
70              _cuotaMensual: '550',
71              _cuotasRestantes: '2',
72              _penalizacionImpago: '10',
73              _ultimoCheckeo: '1702318784'};
74
75          var actualInfo = await prestamo1.methods.mostrarInfo().call({from:
    accounts[1]});
76          assert.equal(expected._cantidadPrestada,actualInfo.
    _cantidadPrestada);
77          assert.equal(expected._state,actualInfo._state);
78          assert.equal(expected._cuotaMensual,actualInfo._cuotaMensual);
79          assert.ok(actualInfo._prestatario);
80          assert.ok(actualInfo._direccion);
81
82
83      });
84
85      it('Chequeo mensual temprano', async() => {
86          try{
87              await prestamo1.methods.checkeoMensual().send({from: accounts
    [0],
88                  gas: web3.utils.toHex(2000000)});
89              assert.fail(new TypeError('No ha pasado tiempo suficiente'));
90          }
91          catch(error){}
92
93      });
94
95      it('Pago Mensual', async() => {
96          await prestamo1.methods.pagarMensualidad().send({from: accounts[1],
97              gas: web3.utils.toHex(2000000), value: web3.utils.toHex(550)});
98          prestamoPagado = await prestamo1.methods.mostrarInfo().call({from:
    accounts[1]});
99
```

```
100         assert.equal(prestamoPagado._cuotasRestantes,1);
101         assert.equal(prestamoPagado._deudaRestante, 550);
102         assert.equal(prestamoPagado._state, 0);
103 
104     });
105 
106     it('Transferir fondos a factory', async() => {
107         //Inversión inicial : 10000 wei
108         //Cantidad prestada : dos préstamos de 1000 wei = 2000 wei
109         //Cantidad devuelta :  Una cuota pagada del prestamo1 = 550 wei
110         //El contrato préstamo debe de tener 0 wei y el contrato factory
    10000-2000+550 = 8550 wei
111 
112         var balanceFactory = await factory.methods.getBalance().call({from:
     accounts[0],
113             gas: web3.utils.toHex(2000000)});
114         assert.equal(balanceFactory,8550);
115     })
116 
117 
118     it('Chequeo mensual', async() => {
119         //Me aseguro que pasa el tiempo suficiente para volver a realizar
    el chequeo
120         await esperar10Segundos();
121         //Realizo chequeo
122         await prestamo1.methods.checkeoMensual().send({from: accounts[0],
123             gas: web3.utils.toHex(2000000)});
124          //Compruebo que el estado del préstamo ha pasado de 'PAGADO'  a '
    ABIERTO'
125         var expectedState = 1;
126         var actualState = await prestamo1.methods.mostrarEstado().call();
127         assert.equal(actualState, expectedState);
128 
129         //Compruebo la penalización del préstamo no pagado
130         var expectedDeudaRestante = 1000 + (1000 * 0.1);
131         var prestamoNoPagado = await prestamo2.methods.mostrarInfo().call({
    from: accounts[2]});
132         assert.equal(prestamoNoPagado._deudaRestante, expectedDeudaRestante)
    ;
133 
134 
135     });
136 
137     it('Eliminación contrato finalizado', async() => {
138         //Pago la última cuota de prestamo1
```

```
139         await prestamo1.methods.pagarMensualidad().send({from: accounts[1],
140             gas: web3.utils.toHex(2000000), value: web3.utils.toHex(550)});
141         prestamoPagado = await prestamo1.methods.mostrarInfo().call({from:
    accounts[1]});
142         //Compruebo que su estado está en finalizado
143         assert.equal(prestamoPagado._state,2);
144         //Realizo el chequeo mensual y este llama a la función de eliminar
145         await esperar10Segundos();
146         await prestamo1.methods.checkeoMensual().send({from: accounts[0],
147             gas: web3.utils.toHex(2000000)});
148 
149         //Compruebo que no existe ni el préstamo finalizado ni su usuario
    el cual no tiene más préstamos en el sistema
150 
151         var vacio = await factory.methods.verContratos(accounts[1]).call({
    from: accounts[1],
152             gas: web3.utils.toHex(2000000)});
153         //No existe contrato para la cuenta del contrato finalizado
154         assert.equal(vacio.length,0);
155 
156 
157         //Solo existe un cliente que es el del contrato no pagado
158         var clientes = await factory.methods.getClientes().call({from:
    accounts[0],
159             gas: web3.utils.toHex(2000000)});
160         assert.equal(clientes.length,1);
161 
162     });
163 
164 
165 
166 });
167 async function esperar10Segundos() {
168     return new Promise(resolve => {
169       setTimeout(() => {
170         resolve('Se puede realizar otro chequeo mensual');
171       }, 10000);
172     });
173   }
```

B.2. test_prestamos_factory.js

Este archivo contiene las pruebas destinadas a verificar el funcionamiento de `PrestamosFactory`.

Listado B.2. Pruebas del contrato PRESTAMOS_FACTORY

```
1  const assert = require('assert');
2  const ganache = require('ganache-cli');
3  const Web3 = require('web3');
4  const web3 = new Web3(ganache.provider());
5  const prestamosFactoryBuild = require ('../build/contracts/PrestamosFactory.
     json');
6  const prestamoCursandoBuild = require ('../build/contracts/PrestamoCursando.
     json');
7
8
9  //Test en distinto proyecto debido al conflicto de versión de web3 y su
     constructor
10 describe('Prestamos factory test', () => {
11
12     before( async() => {
13         accounts = await web3.eth.getAccounts();
14         factory = await new web3.eth.Contract(prestamosFactoryBuild.abi)
15             .deploy({data: prestamosFactoryBuild.evm.bytecode.object})
16             .send({ from: accounts[0], gas: 6721975});
17
18         console.log('contract address:'+ factory.options.address);
19         console.log('Owner: ' + accounts[0]);
20
21
22     } );
23
24     it('deploys a contract', () => {
25         assert.ok(factory.options.address);
26     });
27
28     it('onlyOwner', async () => {
29         try{
30             factory.methods.getBalance().call({from: accounts[1]});
31             assert.fail(new TypeError('Only owner'));
32         } catch (error) {
33         }
```

```
34      });
35
36      it('invertir wei', async () => {
37          await factory.methods.invertir().send({
38              from: accounts[0],
39              value: web3.utils.toHex(10000)
40          });
41          var expected = await web3.eth.getBalance(factory.options.address);
42          assert.equal(expected,10000);
43      });
44
45      it('get Balance', async () => {
46          var expected = await web3.eth.getBalance(factory.options.address);
47          var actual = await factory.methods.getBalance().call({from:
    accounts[0]});
48          assert.equal(expected,actual);
49      });
50
51      it('get empty clientes', async () => {
52          let clientes = await factory.methods.getClientes().call({from:
    accounts[0]});
53          assert.equal(clientes.length,0);
54      });
55
56      it('recolectar fondos', async () => {
57          await factory.methods.recolectarFondos(1000).send({
58              from: accounts[0]
59          });
60          var expected = await web3.eth.getBalance(factory.options.address);
61          assert.equal(expected,9000);
62      });
63
64      it('get empty tipos contrato', async () => {
65          let tiposContrato = await factory.methods.verTiposContrato().call({
    from: accounts[0]});
66          assert.equal(tiposContrato.length,0);
67      });
68
69      it('get empty contratos en curso', async () => {
70          let contratosCursando = await factory.methods.verContratos(accounts
    [0]).call({from: accounts[0]});
71          assert.equal(contratosCursando.length,0);
72      });
73
74      it('get error getting contracts from different account', async () => {
```

```
75        try {
76            let contratosCursando = await factory.methods.verContratos(
  accounts[1]).call({from: accounts[0]});
77            assert.fail(new TypeError('Top Secret'));
78        }
79        catch(error) {}
80
81    });
82
83    it('definir prestamo wrong percentage penalizacion', async() => {
84        try {
85             await factory.methods.definirPrestamo(1000,1000,10,10).send({
  from: accounts[0]});
86             assert.fail(new TypeError('Wrong percentage'));
87        }
88        catch(error) {}
89
90    })
91    it('definir prestamo wrong percentage interes', async() => {
92        try {
93            await factory.methods.definirPrestamo(10,1000,10,10000).send({
  from: accounts[0],
94            gas: web3.utils.toHex(20000)});
95            assert.fail(new TypeError('Wrong percentage'));
96       }
97       catch(error) {}
98
99    });
100
101   it('definir prestamo', async() => {
102       const result = await factory.methods.definirPrestamo(10,1000,2,10).
  send({from: accounts[0],
103           gas: web3.utils.toHex(2000000)});
104       const events = result.events;
105       const eventoDefinido = events['PrestamoDefinido'];
106       assert.ok(eventoDefinido);
107
108   });
109
110   it('get tipos contrato', async () => {
111
112       var expected = {
113           index: '0',
114           porcentajeInteres: '10',
115           cantidad: '1000',
```

```
116             deudaTotal: '1100',
117             cuotas: '10',
118             penalizacionImpago: '10'};
119 
120         let tiposContrato = await factory.methods.verTiposContrato().call({
    from: accounts[0]});
121         assert.equal(tiposContrato.length,1);
122         assert.equal(tiposContrato[0].index, expected.index);
123         assert.equal(tiposContrato[0].deudaTotal, expected.deudaTotal);
124 
125     });
126 
127     it('contratarPrestamo from another account', async() => {
128         try {
129             await factory.methods.contratarPrestamo(accounts[1],0).send({
    from: accounts[2]});
130             assert.fail(new TypeError('Error en dirección'));
131 
132         } catch (error){
133             console.log(error.message)
134 
135         }
136     });
137 
138     it('contratarPrestamo from another account', async() => {
139          await factory.methods.definirPrestamo(10,999999,10,10).send({from:
     accounts[0],
140             gas: web3.utils.toHex(2000000)});
141 
142         try {
143             await factory.methods.contratarPrestamo(accounts[1],1).send({
    from: accounts[1]});
144             assert.fail(new TypeError('No hay suficientes fondos para el
    prestamo'));
145 
146         } catch (error){
147             console.log(error.message)
148         }
149     });
150 
151     it('contratarPrestamo', async() => {
152         var result = await factory.methods.contratarPrestamo(accounts[1],0).
    send({from: accounts[1],
153             gas: web3.utils.toHex(2000000)});
154         var eventoDefinido = result.events['NuevoPrestamoCursando'];
```

```
155         assert.ok(eventoDefinido);
156         console.log(eventoDefinido);
157         result = await factory.methods.contratarPrestamo(accounts[2],0).
    send({from: accounts[2],
158               gas: web3.utils.toHex(2000000)});
159         eventoDefinido = result.events['NuevoPrestamoCursando'];
160         assert.ok(eventoDefinido);
161         console.log(eventoDefinido);
162 
163     });
164 
165     it('verContratos', async() => {
166         var contratos = await factory.methods.verContratos(accounts[1]).
    call({from: accounts[1]});
167         assert.equal(contratos.length,1);
168         contratos = await factory.methods.verContratos(accounts[2]).call({
    from: accounts[2]});
169         assert.equal(contratos.length,1);
170     });
171 
172     it('eliminar contrato no finalizado', async() => {
173         try{
174             await factory.methods.eliminarPrestamoFinalizado(accounts[1],0).
    send({from: contratoEliminar});
175             assert.fail(new TypeError('Deuda pendiente'));
176             } catch (error){
177                 console.log(error.message);
178             }
179     });
180 
181 
182     });
```

Referencias

[1] E. Dans, *«Planteando el bitcoin como alternativa monetaria»*, 2024. dirección: https://www.enriquedans.com/2024/12/planteando-el-bitcoin-como-alternativa-monetaria.html

[2] A. Pascual Sanz, *Repositorio de este proyecto: Sistema de préstamos descentralizado basado en la red Blockchain de Ethereum*, 2024. dirección: https://github.com/AkilinoGit/akilinogit.github.io

[3] A. Pascual Sanz, *Repositorio de pruebas unitarias en mochatest*, 2026. dirección: https://github.com/AkilinoGit/prestamosFactoryMochaTest.git

[4] Aave V3, *Estado del protocolo Aave V3 desplegado en la red*, 2026. dirección: https://etherscan.io/address/0x87870Bca3F3fD6335C3F4ce8392D69350B4fA4E2#code

[5] AAVE, *Documentación del protocolo AAVE*, 2026. dirección: https://aave.com/docs

[6] Compound III, *Estado del protocolo Compound III desplegado en la red*, 2026. dirección: https://etherscan.io/address/0xc3d688B66703497DAA19211EEdff47f25384cdc3#code

[7] Compound, *Documentación del protocolo Compound*, 2026. dirección: https://docs.compound.xyz/

[8] Uniswap V4, *Estado del protocolo Uniswap V4 desplegado en la red*, 2026. dirección: https://etherscan.io/address/0x000000000004444c5dc75cB358380D2e3dE08A90#code

[9] Uniswap, *Documentación del protocolo Uniswap*, 2026. dirección: https://developers.uniswap.org/docs/get-started/concepts/how-uniswap-works

[10] C. Dolader Retamal, J. Bel Roig y J. L. Muñoz Tapia, *«LA BLOCKCHAIN: FUNDAMENTOS, APLICACIONES Y RELACIÓN CON OTRAS TECNOLOGÍAS DISRUPTIVAS»*, 2017. dirección: https://www.mintur.gob.es/Publicaciones/Publicacionesperiodicas/EconomiaIndustrial/RevistaEconomiaIndustrial/405/DOLADER%2C%20BEL%20Y%20MU%C3%91OZ.pdf

[11] A. Bartolomeo y G. Machin Urbay, *«INTRODUCCIÓN A LA TECNOLOGÍA BLOCKCHAIN: SU IMPACTO EN LAS CIENCIAS ECONÓMICAS»*, 2020. dirección: https://bdigital.uncuyo.edu.ar/objetos_digitales/15304/14.-introduccinalatecnologia.pdf

[12] Ethereum.org. «Proof-of-stake (PoS),» visitado 28 de oct. de 2026. dirección: https://ethereum.org/developers/docs/consensus-mechanisms/pos/

[13] Neocheck, *Diagrama del funcionamiento de una blockchain permisionada*, 2019. dirección: https://www.neocheck.es/wp-content/uploads/2019/09/funcionamiento-blockchain-permisionada-800x416.png

[14] Ethereum.org. «Historia de Ethereum: fundador y propiedad.» Sitio web oficial de Ethereum, visitado 21 de oct. de 2026. dirección: https://ethereum.org/es/ethereum-history-founder-and-ownership/

[15] CoinMarketCap, *CoinMarketCap*, 2026. dirección: https://coinmarketcap.com/es/

[16] F. Vogelsteller y V. Buterin, *ERC-20: Token Standard*, Ethereum Improvement Proposals, no. 20, nov. de 2015. dirección: https://eips.ethereum.org/EIPS/eip-20

[17] W. Entriken, D. Shirley, J. Evans y N. Sachs, *ERC-721: Non-Fungible Token Standard*, Ethereum Improvement Proposals, no. 721, ene. de 2018. dirección: https://eips.ethereum.org/EIPS/eip-721

[18] W. Radomski, A. Cooke, P. Castonguay, J. Therien, E. Binet y R. Sandford, *ERC-1155: Multi Token Standard*, Ethereum Improvement Proposals, no. 1155, jun. de 2018. dirección: https://eips.ethereum.org/EIPS/eip-1155

[19] Coinbase, *Coinbase*. dirección: https://www.coinbase.com/es-es/wallet

[20] RabbyWallet, *RabbyWallet*. dirección: https://rabby.io/

[21] Truffle, *Truffle*. dirección: https://archive.trufflesuite.com/

[22] Ganache, *Ganache*. dirección: https://archive.trufflesuite.com/ganache/

[23] Sepolia, *Sepolia*. dirección: https://sepolia.etherscan.io/

[24] Etherscan, *Etherscan*. dirección: https://etherscan.io/

[25] Remix IDE, *Remix IDE*. dirección: https://remix-project.org/?lang=es

[26] Node.js, *Node.js*. dirección: https://nodejs.org/es

[27] Web3.js. «Web3.js documentation,» visitado 28 de oct. de 2026. dirección: https://docs.web3js.org/

[28] Solidity, *Solidity*. dirección: https://soliditylang.org/

[29] Angular, *Angular*, 2026. dirección: https://angular.dev/

[30] Ionic, *Ionic*, 2026. dirección: https://ionicframework.com/

[31] Unión Europea, *Reglamento (UE) 2023/1114, de 31 de mayo de 2023, relativo a los mercados de criptoactivos*, 2023. dirección: https://eur-lex.europa.eu/eli/reg/2023/1114/oj?locale=es

[32] Solidity. «Types: Integers and Division,» visitado 28 de oct. de 2026. dirección: https://docs.soliditylang.org/en/latest/types.html

[33] MetaMask. «Manage user accounts,» visitado 28 de oct. de 2026. dirección: https://docs.metamask.io/metamask-connect/evm/guides/manage-user-accounts/

[34] OECD, «The Limits of DeFi for Financial Inclusion: Lessons from ASEAN,» OECD Publishing, inf. téc., 2024. DOI: 10.1787/f00a0c7f-en dirección: https://doi.org/10.1787/f00a0c7f-en

[35] S. Aramonte, S. Doerr, W. Huang y A. Schrimpf, «DeFi lending: intermediation without information?» Bank for International Settlements, BIS Bulletin 57, jun. de 2022. dirección: https://www.bis.org/publ/bisbull57.pdf

Índice de términos

Glosario

AAVE Protocolo de finanzas descentralizadas (DeFi) de código abierto y no custodio que permite a los usuarios depositar criptoactivos para obtener rendimientos o tomarlos prestados mediante contratos inteligentes sin intermediarios. 6

App Store Tienda digital de aplicaciones. 14

Bitcoin Red blockchain original basada únicamente en transacciones y la que mayor valor ha conservado. 7, 11, 12

blockchain Blockchain es una tecnología de registro distribuido que almacena datos en bloques encadenados de forma segura, inmutable y descentralizada. Se usa principalmente para transacciones digitales, como criptomonedas, pero también tiene aplicaciones en contratos inteligentes, logística y más. 1, 3, 4, 6, 8–10, 12, 14–16, 22, 23, 33, 38, 45, 58

cartera Software o hardware que permite gestionar claves y firmar transacciones con tus criptomonedas sin depender de un intermediario como un banco. 15, 16, 61

contrato inteligente Programa o código autoejecutable que se ejecuta en una blockchain. Define reglas y condiciones preestablecidas y, cuando se cumplen, ejecuta automáticamente acciones sin necesidad de intermediarios. Se usa en transacciones, acuerdos digitales y aplicaciones descentralizadas. 1–4, 6, 12–19, 21, 27–29, 31–33, 36–38, 43, 45, 50, 52, 58, 88

criptomoneda Activo digital que utiliza técnicas criptográficas y una red blockchain para registrar y verificar transacciones sin necesidad de una autoridad central. Puede emplearse como medio de intercambio. 14, 16

dApp Decentralized App o aplicación descentralizada, es una aplicación que funciona sobre una red descentralizada de blockchain en lugar de un servidor central. Uti-

liza contratos inteligentes para gestionar su lógica y datos. 4, 14–16, 19, 27, 28, 39, 47

DeFi Decentralized Finance: Sistema financiero en un entorno descentralizado, como puede ser una red blockchain. 2–6, 14, 15, 19, 58, 61, 63

Diagrama de secuencia UML Diagrama que muestra cómo interactúan los objetos de un sistema en un orden temporal, representando el intercambio de mensajes entre ellos. 23

Diagrama-UML Un diagrama UML (Unified Modeling Language) es una representación gráfica utilizada para modelar, visualizar, documentar y diseñar sistemas de software. 22

entorno de trabajo Conjunto de herramientas, bibliotecas y convenciones diseñadas para facilitar el desarrollo de aplicaciones, proporcionando una estructura base que simplifica y organiza el código. 8

ETH Ether es la criptomoneda de la red Ethereum. 1, 7, 14, 15, 19, 21, 27, 30, 43, 45, 49

Ethereum Red blockchain cuya moneda principal es el ETH y es capaz de ejecutar contratos inteligentes. 1–3, 6, 7, 12, 13, 15, 16, 18, 19, 31, 35, 40, 45, 58, 88

Euribor Es el tipo de interés promedio al que los principales bancos de la zona euro se prestan dinero entre sí en el mercado interbancario. Se calcula diariamente y sirve como referencia para numerosos productos financieros, especialmente para hipotecas a tipo variable en Europa. 26

fondo de liquidez Grupo de fondos disponibles para prestar en una plataforma DeFi. 6, 7

gas Unidad que mide el trabajo computacional necesario para ejecutar operaciones en Ethereum. La comisión se paga en ETH y depende del gas consumido y del precio por unidad de gas. 3, 22, 41, 57

hash Un hash es un valor único generado por una función hash a partir de un conjunto de datos de entrada. Se utiliza para representar esos datos de manera compacta y segura. 8–11

hashing Proceso de realizar un hash. 11

inicio de sesión El inicio de sesión es el proceso de autenticación mediante el cual un usuario se identifica para acceder a un sistema, aplicación, sitio web o dispositivo informático de forma segura. 26

JavaScript JavaScript es un lenguaje de programación interpretado, ligero y versátil, utilizado principalmente para crear contenido web interactivo. 17

MetaMask MetaMask es una extensión de navegador que permite a los usuarios interactuar con aplicaciones descentralizadas (DApps) en la red Ethereum, facilitando la gestión de cuentas y la firma de transacciones. 19

NFT Token no fungible: activo digital distinguible de otros tokens por un identificador propio. No es intercambiable de forma equivalente con cualquier otro token. El estándar ERC-721 permite registrar su titularidad y transferirlo; su relación con un activo externo depende del sistema que lo utilice [17]. 15, 59

Oráculo Un oráculo en blockchain es un servicio que proporciona datos externos a los contratos inteligentes, permitiéndoles interactuar con información del mundo real, como precios, clima o eventos deportivos. 60

Pop-up Ventana emergente que aparece sobre la interfaz de una aplicación o página web para mostrar información o solicitar una acción del usuario, como la autorización de una transacción. 15

script Conjunto de instrucciones o código escrito en un lenguaje de programación o de scripting que se ejecuta automáticamente para realizar tareas específicas. 12

SHA-256 Secure Hash Algorithm 256 es una función hash criptográfica que genera un valor único de 256 bits (32 bytes) a partir de una entrada. 11

Solidity Lenguaje de programación utilizado para programar contratos inteligentes. 14, 16

Tipo Address El tipo address representa direcciones de la red. 35

Token Activo digital emitido en una red blockchain que representa un valor, un derecho o una utilidad específica dentro de una plataforma. 14

UI User Interface: Interfaz de usuario. 8

Uniswap Bolsa de intercambio descentralizada basada en Ethereum que automatiza el intercambio de criptomonedas mediante fondos de liquidez, sin intermediarios ni libros de órdenes tradicionales. 7

Web3 Web3 es la evolución de la web que utiliza tecnología blockchain para descentralizar aplicaciones y servicios, permitiendo mayor control, privacidad y propiedad de los datos por parte de los usuarios. 4, 15, 17

wei El wei es la unidad más pequeña de Ether (ETH), donde 1 ETH equivale a 10^{18} wei. Es la unidad de medida utilizada en las transacciones de la blockchain de Ethereum. 41, 48–50

Siglas

DNI Documento Nacional de Identidad. 14

EVM Ethereum Virtual Machine, Máquina Virtual de Ethereum. 13–18

OCDE Organización para la Cooperación y el Desarrollo Económico. 60

ODS Objetivos de Desarrollo Sostenible. 62, 63

OP Objetivo principal. 2, 45, 48

OS Objetivo secundario. 2, 3, 45, 46, 48

PFG proyecto de fin de grado. 4

PoS Proof of stake, Prueba de participación. 10, 61, 63

PoW Proof of Work, Prueba de trabajo. 10, 61, 63

